Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Universidad Nacional de Rosario

Redes adversarias generativas para el reconocimiento de acciones en videos deportivos

Luciano N. Chehem

Director:
Dr. Guillermo L. Grinblat

Codirector:
Dr. Lucas C. Uzal

Tesina de Grado
Licenciatura en Ciencias de la Computación

Diciembre de 2018

Resumen

Las redes neuronales convolucionales son una poderosa clase de modelos para la resolución de problemas de reconocimiento de imágenes. En particular, las redes adversarias generativas (GAN) mostraron resultados muy prometedores en los últimos años, debido a su capacidad para generar imágenes fotorrealistas. Incentivados por estos resultados, proponemos clasificar acciones en videos deportivos utilizando este tipo de redes. Para ello, tomaremos como entrada de la red un conjunto de imágenes (frames de videos) en lugar de uno solo, como ocurre en las configuraciones usuales de GAN. Esto quiere decir que cada muestra contendrá tres canales por cada frame, en lugar de tres canales en total. Como subproducto, en el proceso de entrenamiento se genera una subred, que toma como entrada imágenes y reduce sus dimensiones espaciales hasta obtener un vector, el cual es llamado vector de representación. Utilizaremos estos vectores en recortes de videos junto a modelos SVM para lograr nuestro objetivo. Alternativamente, usaremos la misma red aplicada a la imagen completa y le acoplaremos una capa softmax en su salida, buscando resolver el mismo problema. También realizaremos pruebas inicializando la red con los pesos aprendidos en el entrenamiento de la red GAN y, por otro lado, inicializándolos de manera aleatoria. Todas las pruebas serán realizadas utilizando diferentes combinaciones de tamaños de imágenes y cantidades de frames en sus muestras. Los resultados indican que el enfoque con SVMs es superado por el que utiliza una red convolucional con una capa softmax. A su vez, este último mejora aún más al utilizar como preentrenamiento los pesos aprendidos por la red GAN, obteniendo mejores resultados de clasificación en casi todos los casos.

Agradecimientos

A mis padres María Teresa y Rubén por su apoyo incondicional, no sólo durante esta etapa de mi vida, sino desde siempre. Sin su apoyo, su paciencia conmigo antes de cada final y sus palabras de aliento hubiera sido imposible llegar hasta acá. A mi hermana Marianela por ayudarme con la carrera, sobre todo al comienzo de la misma, y por ser a quien primero suelo acudir en busca de ayuda o de una opinión sabia y sincera. A mi compañera de vida Andreina, quien me brindó en cada momento todo su apoyo y amor, haciendo que este año sea hermoso a pesar de los momentos oscuros. A mi tía Marcela, sin su ayuda hubiera sido mucho más difícil dedicar el tiempo y esfuerzo necesario para terminar este trabajo. A los amigos que hice mientras cursaba, que hicieron infinitamente más fácil preparar cada materia, con quienes ahora comparto gratos momentos fuera de la facultad. A mis directores Guille y Lucas por su invaluable ayuda durante mi etapa final como estudiante, por su infinita paciencia y el valioso tiempo que invirtieron conmigo. A todos ellos muchísimas gracias.

Índice general

1. Introducción **6**
1.1. Objetivos 7
1.2. Organización del trabajo 7

2. Conceptos generales de aprendizaje automatizado **9**
2.1. Clasificación 11
2.1.1. Error de clasificación y *accuracy* 11
2.2. Sesgo inductivo, selección de modelos y generalización 12
2.3. Sobreajuste 12
2.4. Conjuntos de entrenamiento, validación y prueba 13
2.5. Reducción de dimensionalidad 13
2.5.1. Principal component analysis (PCA) 14
2.6. Clustering 15
2.6.1. k-means 15
2.7. Algoritmos de clasificación 16
2.7.1. Support vector machines 16
2.7.1.1. Kernels 19
2.7.1.2. Clasificación multiclase 20
2.7.2. Redes neuronales 21
2.7.2.1. Neuronas 21
2.7.2.2. Perceptrón multicapa 23
2.7.2.3. Capa softmax 23
2.7.2.4. Descenso por gradiente 24
2.7.2.5. Modificaciones del algoritmo de descenso por gradiente 25
2.7.2.5.1. Descenso de gradiente estocástico . . 26
2.7.2.5.2. SGD de minilote 26
2.7.2.5.3. Momento lineal (*momentum*) 26

2.7.2.5.4. *Learning rates* adaptables por parámetro . . . 27
2.7.2.6. Entrenamiento con *backpropagation* . . . 28
2.7.3. Redes neuronales profundas . . . 28
2.7.3.1. Redes convolucionales . . . 29
2.7.3.1.1. Arquitecturas de redes convolucionales 36
2.7.4. Redes adversarias generativas . . . 36

3. Experimentación 39
3.1. Datasets . . . 41
3.1.1. Rugby no etiquetado . . . 41
3.1.2. Rugby etiquetado . . . 42
3.1.3. Preprocesamiento de los datos . . . 43
3.1.3.1. Campo visual de ventanas . . . 43
3.1.3.2. Duraciones de muestras . . . 45
3.1.3.3. Dataset U: Rugby no etiquetado, por ventanas 46
3.1.3.4. Dataset L: Rugby etiquetado . . . 46
3.1.3.4.1. Dataset L1: Rugby etiquetado, por ventanas . . . 47
3.1.3.4.2. Dataset L2: Rugby etiquetado, región completa . . . 50
3.1.3.4.2.1. Relación de aspecto . . . 50
3.2. Experimentos . . . 51
3.2.1. Entrenamiento de DCGAN . . . 52
3.2.1.1. Construcción del modelo . . . 52
3.2.1.1.1. Generador . . . 53
3.2.1.1.2. Discriminador . . . 54
3.2.1.1.3. Implementación . . . 54
3.2.1.2. Entrenamiento . . . 55
3.2.1.3. Generación de nuevos datos de la red . . . 55
3.2.1.3.1. Inconvenientes durante el entrenamiento . . . 55
3.2.1.4. Extracción del discriminador . . . 57
3.2.1.5. Previsualización de resultados mediante PCA e histogramas . . . 58
3.2.1.5.1. Resultados del *clustering* sobre vectores representación . . . 59

3.2.1.5.1.1. Clusters encontrados con k-means . . . 59
3.2.1.5.1.2. Clusters definidos por histogramas . . . 59
3.2.1.5.2. Resultados del *clustering* sobre píxeles 61
3.2.2. Predicción con DCGAN + SVMs . . . 61
3.2.2.1. Ejecución de red discriminador sobre datos . . 66
3.2.2.2. Support vector machines . . . 66
3.2.2.2.1. Preprocesamiento de los datos . . . 66
3.2.2.2.2. Construcción de los modelos . . . 67
3.2.2.2.2.1. Implementación . . . 68
3.2.2.2.3. Entrenamiento . . . 68
3.2.2.2.3.1. Tiempos de entrenamiento . . 69
3.2.2.2.4. Resultados . . . 69
3.2.2.2.4.1. Comparación por tamaño de campo visual y cantidad de frames . . . 70
3.2.2.2.4.2. Modelo seleccionado . . . 70
3.2.3. Predicción con redes convolucionales . . . 71
3.2.3.1. Construcción del modelo . . . 72
3.2.3.1.1. Convoluciones de la red . . . 73
3.2.3.1.2. Implementación . . . 74
3.2.3.2. Entrenamiento . . . 74
3.2.3.3. Resultados . . . 74
3.2.4. Predicción con redes convolucionales preentrenadas con DCGAN . . . 75
3.2.4.1. Construcción del modelo . . . 75
3.2.4.1.1. Implementación . . . 76
3.2.4.2. Entrenamiento . . . 76
3.2.4.3. Resultados . . . 76
3.2.5. Generalización a videos completos . . . 78
3.2.5.1. Método con votación simple . . . 80
3.2.5.2. Métodos con votación ponderada por ubicación temporal del fragmento . . . 81
3.2.5.2.1. Ajuste . . . 81
3.2.5.3. Resultados . . . 83

4. Discusión y conclusiones **84**

5. Trabajos futuros **88**

Bibliografía **89**

A. Generación de imágenes artificiales **94**
A.1. Calidad de las imágenes 94
A.2. Movimiento de las imágenes 97
A.3. Entrenamiento con pérdida de contraste 101

B. Agrupamiento de imágenes utilizando vectores representación **104**

Capítulo 1

Introducción

En los últimos años se ha extendido el uso de GANs (generative adversarial networks) [1], debido principalmente a su capacidad de generar imágenes fotorrealistas. Esto se logró combinando dicha técnica con redes convolucionales profundas, dando origen a las redes neuronales conocidas como DCGANs (deep convolutional generative adversarial networks) [2].

Sin embargo, estas técnicas fueron aplicadas en su gran mayoría a imágenes estáticas, debido probablemente a la diferencia entre volúmenes de datos que se deben manejar cuando se trabaja con imágenes y cuando se hace con videos.

Debido a su buen desempeño en el campo de las imágenes [3, 4, 5, 6, 7], se utilizan DCGANs para capturar información proveniente del movimiento que puede percibirse entre frames y no sólo de la información que se obtiene al analizar un frame a la vez.

Para ello, se utiliza como caso de prueba la problemática del reconocimiento y clasificación de videos deportivos, en particular videos que contienen jugadas de rugby.

Este caso de prueba tiene especial utilidad para entrenadores de rugby (y otros deportes) que brinden charlas técnicas a los deportistas que dirigen. Para ello, es de utilidad contar con material en video, ya clasificado y rápidamente accesible para, por ejemplo, obtener un compilado de jugadas similares, o un partido completo etiquetado con los diferentes tipos de acciones que ocurrieron y así poder saltar de una a otra jugada.

Otro uso posible para los métodos desarrollados en este trabajo es el de etiquetar automáticamente filmaciones profesionales de eventos deportivos, para luego crear un resumen con las jugadas más importantes de un partido o

brindar a sus televidentes una forma de acceder inmediatamente a las jugadas de determinado tipo. Por ejemplo, ver todos los lines de un partido de rugby, o llevado a otro deporte, ver todos los penales efectuados en un campeonato.

1.1. Objetivos

El objetivo general de este trabajo consiste en estudiar técnicas de aprendizaje semisupervisado basadas en GANs para la clasificación de videos deportivos, para ello:

- Utilizando GANs, se genera una representación común para secuencias de fotogramas. El entrenamiento es realizado utilizando videos no etiquetados, es decir que es mediante aprendizaje no supervisado.
- Se utilizan SVMs en conjunto con dichas representaciones para clasificar jugadas de rugby contenidas en fragmentos de videos.
- Se continúa de manera supervisada con el entrenamiento del discriminador, el cual fue entrenado mediante GAN en el primer objetivo. Para ello se modifica la estructura de la última capa del mismo, convirtiéndolo en un clasificador multiclase, que es utilizado para clasificar jugadas en fragmentos de videos.

En la primera etapa, en la cual se utilizan muestras no etiquetadas, se dispone de un conjunto de datos varias veces mayor que en las etapas siguientes, donde se requieren videos con jugadas etiquetadas. Un punto interesante de las técnicas utilizadas en este trabajo es que su objetivo es aprovechar la gran cantidad de datos no etiquetados (muy fácilmente obtenibles) para hacer un mejor uso de los datos etiquetados, de los cuales se precisa una cantidad mucho menor. En general, estos últimos tipos de muestras son mucho más difíciles de obtener, debido a que casi siempre se requiere de trabajo manual para generarlas.

1.2. Organización del trabajo

El presente trabajo está organizado de la siguiente manera:

- En el Capítulo 2 se introducen todos los conceptos propios del aprendizaje automatizado necesarios para entender el trabajo, junto con algunos algoritmos utilizados en el mismo.
- En el Capítulo 3 se detallan los datasets con los que se trabaja, junto con los preprocesamientos que se les aplica para producir los datos de entrada de los modelos utilizados en las diferentes partes del trabajo. Luego se describen los diferentes algoritmos utilizados durante la experimentación, junto con su construcción y detalles de implementación, los entrenamientos realizados a dichos modelos y finalmente los resultados obtenidos con cada uno de ellos.
- En el Capítulo 4 se discuten y analizan los resultados y se exponen las conclusiones a las que se llega.
- En el Capítulo 5 se proponen posibles trabajos futuros.

Capítulo 2

Conceptos generales de aprendizaje automatizado

El aprendizaje automatizado es una rama de las ciencias de la computación, más específicamente de la inteligencia artificial, que explora la construcción y el estudio de algoritmos que pueden "aprender" de los datos y hacer predicciones sobre éstos. Dichos algoritmos operan construyendo un modelo en base a ejemplos, para hacer predicciones o tomar decisiones, en lugar de ejecutar instrucciones estrictamente estáticas o algoritmos basados en reglas.

Es empleado en un rango de tareas para las cuales diseñar y programar algoritmos explícitos con buen desempeño es difícil o no es factible, como por ejemplo detección de spam, visión por computadora, detección de intrusos en redes, cálculo de riesgos, etcétera.

Todo proceso de aprendizaje consiste de dos etapas: la estimación de dependencias desconocidas en un sistema de un conjunto dado de datos y el uso de esas dependencias estimadas para predecir nuevas salidas del sistema.

De acuerdo a Russell y a Norvig [8], si se los clasifica considerando el tipo de *feedback* o devolución que se le da al algoritmo durante el entrenamiento, existen tres tipos comunes de métodos de aprendizaje automatizado, a los cuales se les suma un cuarto, con características comunes a dos de estos tres:

Aprendizaje supervisado En este tipo de aprendizaje, un conjunto etiquetado de datos de entrenamiento (ejemplos) es utilizado para inferir una función que mapea los datos de entrada a la salida deseada. Cada ejemplo es un par formado de un objeto de entrada (típicamente un vector) y un

valor de salida deseado. Un algoritmo de aprendizaje supervisado analiza los datos de entrenamiento y produce una función inferida, que puede ser utilizada para mapear nuevos ejemplos. Un escenario óptimo permitirá al algoritmo determinar correctamente las etiquetas de clase para instancias nunca vistas. Esto requiere que el algoritmo de aprendizaje generalice a partir de los datos de entrenamiento a situaciones no vistas de una forma "razonable" (ver 2.2).

Aprendizaje no supervisado En contraste, bajo los términos del aprendizaje no supervisado, el algoritmo aprende patrones en la entrada pese a que no es provista una devolución explícita, es decir que no se utilizan ejemplos etiquetados y no hay ninguna información sobre la salida deseada durante el proceso de aprendizaje. Como resultado de ello, es tarea del modelo encontrar patrones o descubrir los grupos de los datos de entrada. El algoritmo deberá inferir una función que describa la estructura de datos no etiquetados (datos que no han sido clasificados o categorizados). Dado que los ejemplos brindados al algoritmo de aprendizaje son no etiquetados, no hay una manera directa para evaluar la precisión de la estructura que es producida por el algoritmo, una característica que lo separa del aprendizaje supervisado. La tarea más común en este tipo de aprendizaje es el *clustering* (agrupamiento), que consiste en detectar grupos de datos (clústeres) potencialmente útiles en los datos de entrada.

Aprendizaje por refuerzo En este caso, el algoritmo aprende de una serie de refuerzos (recompensas o castigos). Corresponde al algoritmo decidir cuáles de sus acciones previas al refuerzo fueron responsables de éste y en qué medida. Por ejemplo, la falta de propina al final de un viaje le indica a un algoritmo de conducción de taxis que hizo algo incorrecto. Los puntos obtenidos por ganar una partida le indicarían a un algoritmo jugador de ajedrez que hizo algo correcto.

Aprendizaje semisupervisado La adquisición de datos etiquetados para resolver un problema de aprendizaje automatizado usualmente requiere de un agente humano calificado (por ejemplo, para transcribir segmentos de audio) o de experimentos físicos (por ejemplo, realizar mediciones). El costo asociado al proceso de etiquetar datos puede volver imposible la obtención de un dataset completamente etiquetado, mientras que la adquisición de datos no etiquetados suele ser una tarea mucho más simple. El aprendizaje semisupervisado consiste en una clase de tareas y técnicas

de aprendizaje supervisado en las cuales también se hace uso de datos no etiquetados para el entrenamiento, típicamente una pequeña cantidad de datos etiquetados junto con una gran cantidad de datos no etiquetados.

2.1. Clasificación

La tarea de clasificación consiste en asignar cada vector de entrada a una de un número finito de categorías discretas [9].

Sean n la cantidad de atributos (*features*) que describen a los objetos con los que se trabajará y c la cantidad de clases o categorías a las que pueden ser asignados dichos objetos. El conjunto de datos de entrenamiento T entonces será un subconjunto de

$$\{\ (X,\ l) \mid X \in \mathbb{R}^n \wedge l \in \{\, 0,\ ...,\ c-1 \,\}\ \}.$$

El algoritmo utilizará los datos de T para inferir una función clasificadora

$$h : \mathbb{R}^n \longmapsto \{\, 0,\ ...,\ c-1 \,\}.$$

2.1.1. Error de clasificación y *accuracy*

Dado un conjunto de datos T, el error de clasificación del modelo h se define como

$$error(h) = N_{err}/N_{tot}\,,$$

donde N_{err} es el número de clasificaciones erróneas de h en T y N_{tot} es el total de datos clasificados. Formalmente, sea $h(x_i)$ la etiqueta o clase asignada por h a un objeto $x_i \in T$, y sea $l(x_i)$ la clase real correspondiente a ese objeto, entonces

$$error(h) = \frac{1}{N_{tot}} * \sum_{i=1}^{N_{tot}} \phi(h(x_i), l(x_i)),$$

donde

$$\phi(a,b) = \begin{cases} 1, & \text{si } a \neq b \\ 0, & \text{si } a = b \end{cases}.$$

Con el error ya definido, es fácil definir la *accuracy* (exactitud) de un modelo sobre un conjunto de datos como

$$accuracy(h) = 1 - error(h).$$

2.2. Sesgo inductivo, selección de modelos y generalización

A medida que vemos más ejemplos de entrenamiento, conocemos más sobre la función subyacente y podemos eliminar más hipótesis que no son consistentes de nuestro conjunto de hipótesis, pero aún podrían quedar muchas hipótesis consistentes.

Es por esto que debemos hacer algunas suposiciones adicionales para llegar a una única solución con los datos que tenemos. El conjunto de suposiciones que hacemos para hacer posible el aprendizaje es llamado sesgo inductivo del algoritmo.

Sabemos que cada conjunto de hipótesis posibles tiene una cierta capacidad y puede aprender solamente ciertas funciones. La clase de funciones factibles de ser aprendidas puede extenderse al utilizar un conjunto mayor de hipótesis posibles, conteniendo hipótesis más complejas. Por ejemplo, si estamos ante un problema de regresión, al utilizar un polinomio de mayor grado estaremos aumentando la capacidad de nuestra función, pero sus hipótesis serán más complejas.

La pregunta ahora es cuándo debemos detenernos. Debido a que el aprendizaje no es posible sin sesgo inductivo, es importante cómo elegir el sesgo correcto. Esto es llamado selección del modelo. Al responder esta pregunta, debemos recordar que el objetivo del aprendizaje automatizado no es replicar los datos de entrenamiento, sino predecir nuevos casos. Entonces quisiéramos ser capaces de generar predicciones correctas para casos fuera del conjunto de entrenamiento.

La capacidad de clasificar correctamente nuevos ejemplos que difieren de aquellos utilizados para el entrenamiento es conocida como generalización [9].

2.3. Sobreajuste

Generalmente, a medida que incrementamos la complejidad de nuestras hipótesis, el error de entrenamiento decrece, ya que nuestro modelo puede adaptarse con mayor precisión a los datos de entrenamiento. Pero si tenemos un conjunto muy complejo de hipótesis posibles, los datos pueden no ser suficientes para elegir la mejor hipótesis. Esto es conocido como sobreajuste.

Para evitar el sobreajuste, una posibilidad es la de aumentar el tamaño del conjunto de entrenamiento, pero eso sólo puede servir hasta cierto punto.

Dado un conjunto de entrenamiento y un conjunto de hipótesis posibles, podemos encontrar la hipótesis dentro de este conjunto que tenga el menor error de entrenamiento. Sin embargo, si el conjunto de hipótesis posibles no está bien elegido, no importa qué hipótesis elijamos, no tendremos una buena generalización.

2.4. Conjuntos de entrenamiento, validación y prueba

Cuando se analiza un modelo se puede medir su habilidad de generalización, es decir, la calidad de su sesgo inductivo, si se tiene acceso a datos fuera del conjunto de entrenamiento. En la práctica, esto se simula dividiendo el conjunto de entrenamiento en dos partes.

Una parte se utiliza para entrenar (es decir, para ajustar parámetros del modelo), y la otra parte, llamada conjunto de validación, es utilizada para medir su capacidad de generalización.

Entonces, dado un conjunto de clases de hipótesis posibles H_i, para cada una de ellas, se busca la mejor hipótesis $h_i \in H_i$ para el conjunto de entrenamiento. Después, asumiendo conjuntos lo suficientemente grandes de entrenamiento y validación, la hipótesis que es más certera sobre el conjunto de validación será la mejor (la que tiene una mejor generalización).

Es muy importante destacar que si se necesita calcular el error para tener una idea sobre el error esperado del mejor modelo, no se deben utilizar datos del conjunto de validación. El conjunto de validación es utilizado para elegir el mejor modelo, por lo que efectivamente se vuelve parte del conjunto de entrenamiento. Es por ello necesario utilizar un tercer conjunto, llamado conjunto de prueba, el cual contiene ejemplos que no han sido utilizados en entrenamiento ni validación.

2.5. Reducción de dimensionalidad

La complejidad de cualquier clasificador es afectada por el número de entradas. Esto determina tanto la complejidad temporal como espacial y el número de ejemplos de entrenamiento necesarios. Al enfrentar este problema, se utilizan métodos para disminuir la cantidad de componentes o dimensiones de un conjunto de datos, ya sea eligiendo un subconjunto con los com-

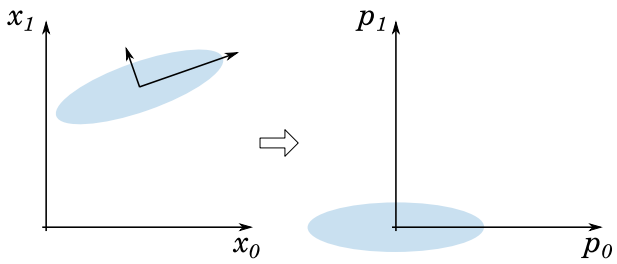


Figura 2.1: Ejemplo de PCA con dos dimensiones. El procedimiento centra la muestra y rota los ejes para alinearlos con las direcciones de mayor varianza. En el eje p_0 se encuentra el primer componente principal y en p_1 el segundo. Fuente: E. Alpaydin (2014) [10].

ponentes más importantes y podando el resto (selección de componentes), o creando nuevos componentes (y en menor cantidad) a partir de los originales (extracción de componentes).

2.5.1. Principal component analysis (PCA)

Un algoritmo de extracción de componentes es el llamado análisis de componentes principales. Para ello, encuentra un mapeo desde sus entradas en el espacio d-dimensional original a un nuevo espacio ($k \leq d$)-dimensional, que tiene mínima pérdida de información.

Es no supervisado, ya que no utiliza la información de salida, sino que maximiza la varianza de los datos de entrada.

El componente principal w_1 es uno tal que las muestras, luego de ser proyectadas sobre este componente, se encuentran más esparcidas, haciendo que la diferencia entre los puntos sea más evidente. Para ello se busca el vector w_1 que maximiza la varianza.

Los siguientes componentes se obtienen con las mismas restricciones y cada uno debe ser ortogonal a todos los anteriores. Esto es para que los datos, luego de proyectarse sobre dos componentes diferentes, no estén correlacionados.

El componente principal explica la mayor parte de la varianza, el segundo explica la segunda parte más grande, y así sucesivamente.

En la Figura 2.1 se observa un ejemplo muy simple de PCA para dos dimensiones.

2.6. Clustering

El *clustering* (o clusterización) es un problema de aprendizaje no supervisado, es decir que se tienen los ejemplos sin etiquetas. Consiste en encontrar grupos de ejemplos similares en los datos [9].

Formalmente, dado un conjunto de datos $X = \{x_0, x_1, ..., x_n\}$ y alguna noción de similaridad $s_{ij} > 0$ entre los pares de puntos x_i y x_j, se define un k-clúster como una partición de X en k conjuntos no vacíos, tal que las similaridades entre los datos dentro del mismo conjunto sean altas mientras que la de los datos entre diferentes grupos sean bajas.

Dado un k-clúster, opcionalmente se le puede asignar un significado a cada clúster, o puede utilizarse sin interpretaciones adicionales.

2.6.1. *k*-means

El algoritmo k-means es utilizado para encontrar grupos en los datos, donde los grupos son representados por sus centroides [10].

Sea $X = \{x_0, x_1, ..., x_{n-1}\}$ un conjunto de datos formado por n observaciones de una variable aleatoria d-dimensional x. El objetivo es particionar el conjunto de datos en un número k de clústers, donde supondremos que el valor de k es conocido.

Intuitivamente, podemos pensar que un clúster comprende un grupo de puntos cuyas distancias entre uno y otro son pequeñas comparadas con las distancias a puntos fuera del clúster. Esta noción puede formalizarse introduciendo primero un conjunto de vectores d-dimensionales μ_k, con $k \in \{0, 1, ..., k-1\}$, donde μ_k es un prototipo asociado al k-ésimo clúster. Se puede pensar a μ_k como el punto que representa el centroide del clúster. El objetivo de k-means es, entonces, encontrar una asignación de puntos de los datos a clústers, así como un conjunto de vectores $M = \{\mu_0, \mu_1, ..., \mu_{k-1}\}$, tales que la suma de los cuadrados de las distancias de cada punto a su vector μ_k más cercano es mínima [9].

En la Figura 2.2 se observa un ejemplo muy simple de k-means para dos dimensiones.

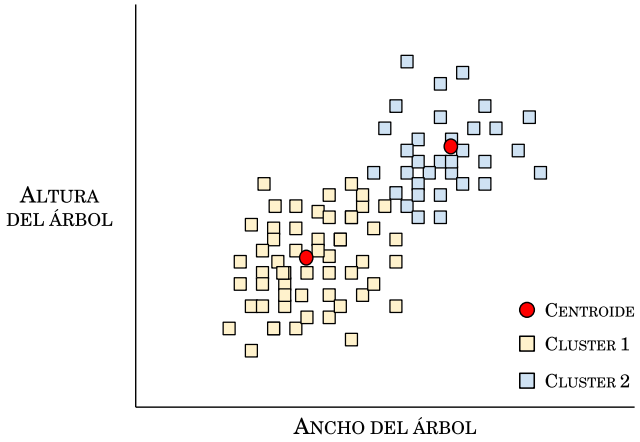


Figura 2.2: Ejemplo de k-means con dos dimensiones: altura y ancho del árbol. Los dos clústers encontrados podrían etiquetarse como "árboles enanos" y "árboles grandes". Fuente: Google LLC (2018) [11].

2.7. Algoritmos de clasificación

En esta sección se presentan y describen los clasificadores utilizados en este trabajo.

2.7.1. Support vector machines

Las máquinas de vectores soporte, o support vector machines (SVM) [12], [13] son un método de aprendizaje supervisado relativamente reciente, desarrollado por Vladimir Vapnik.

En su forma más básica resuelven problemas lineales de clasificación binaria, pero agregando la noción de *kernel* es posible trabajar con clases que no son linealmente separables, y ensamblando varios modelos se logra la clasificación multiclase. También pueden aplicarse a problemas de regresión.

La idea detrás de las SVM es separar los puntos de cada clase mediante un hiperplano. Como una simplificación de los problemas a resolver, veamos el caso en que nos encontramos trabajando en un espacio de dos dimensiones, en el cual los puntos de entrenamiento son separables por al menos una recta. El problema es que puede haber más de una, como se ilustra en la Figura 2.3, por lo que el problema se trata, no sólo de encontrar una recta que separe las clases, sino encontrar la que mejor lo haga.

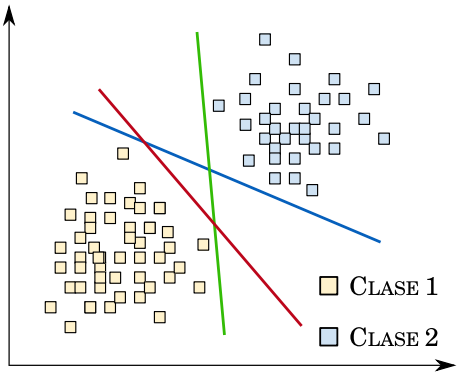


Figura 2.3: Muestras de datos separables por más de una recta. Las tres rectas graficadas separan sin error todos los ejemplos.

Si definimos a M (margen) como la distancia entre la recta y los puntos más cercanos a ella de cada clase, podemos elegir la recta maximizando M, es decir, la recta que se encuentra más lejos de los puntos más cercanos de cada clase, como se observa en la Figura 2.4.

Sin embargo, en los problemas reales es raro que exista una recta que separe perfectamente los puntos de ambas clases de manera que ningún punto quede del lado incorrecto de la misma (Figura 2.5). Por lo tanto, es razonable permitir que algunos puntos del conjunto de entrenamiento no sean correctamente clasificados, para lo cual se define un hiperparámetro de penalización comúnmente llamado C.

Este parámetro le indica al proceso de optimización de SVM cuánto error es tolerable. De esta forma se busca encontrar un equilibrio entre valores mal clasificados durante el entrenamiento y el margen M. Valores grandes de C crean una recta con un margen pequeño al aumentar la penalización recibida por los puntos mal clasificados, mientras que valores pequeños encuentran rectas con márgenes mayores, a costa de aumentar la cantidad de ejemplos mal clasificados, pero buscando obtener una mejor generalización.

Para formalizar lo explicado anteriormente (sin incluir el hiperparámetro de penalización), diremos que cualquier hiperplano puede definirse como los puntos de $\vec{x}$ que satisfacen $\vec{w} \cdot \vec{x} - b = 0$, donde $\vec{w}$ es un vector normal al mismo. Luego, podemos poner como condición que todos los puntos de una clase satisfagan la inecuación

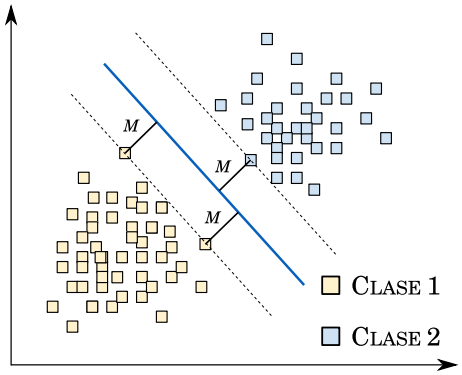


Figura 2.4: Recta que separa los ejemplos con el mayor margen posible. Los puntos en contacto con las líneas punteadas (márgenes) son los llamados "vectores soporte", que le dan nombre al método.

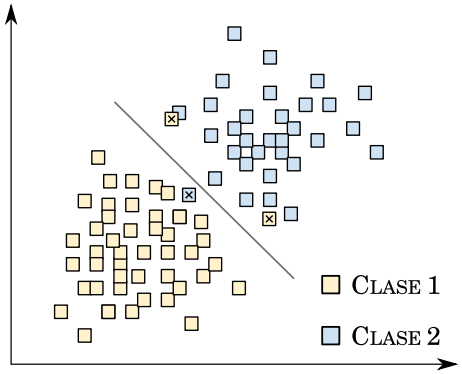


Figura 2.5: Ejemplo con tres puntos que no son correctamente clasificados por la recta escogida. Además, en este caso particular no existe ninguna recta que separe completamente las dos clases, ya que hay una muestra de la Clase 2 que se encuentra dentro del polígono más pequeño que contiene a todas las muestras de la Clase 1.

$$\vec{w} \cdot \vec{x}_i - b \geq 1$$

y que los puntos pertenecientes a la otra clase satisfagan la inecuación

$$\vec{w} \cdot \vec{x}_i - b \leq -1.$$

Esto significa que los puntos de una de las clases se encuentran de un lado de un plano, y los puntos de la otra se encuentran del lado opuesto a un plano paralelo al primero. Ahora, podemos hacer una búsqueda en el espacio de $\vec{w}$ y b para encontrar los parámetros que maximizan el margen mientras que continúan clasificando correctamente todos los puntos. Sin entrar en detalles, diremos que para ello es posible plantear este problema de minimización y solucionarlo a través de su dual [8].

2.7.1.1. Kernels

En algunos casos, un hiperplano puede no ser suficiente para separar los datos, debido a que los puntos se encuentran distribuidos en formas irregulares, haciendo que la frontera entre una clase y otra sea compleja y no pueda representarse con un hiperplano.

Para estos casos, una solución utilizada frecuentemente junto a SVMs consiste en mapear los datos de su espacio original a algún espacio de mayor dimensionalidad, donde las clases sí sean separables. De esta forma el problema no-lineal original se puede resolver con métodos lineales.

Este mapeo se realiza con una función llamada *kernel*. En la Figura 2.6 se presenta un ejemplo muy sencillo donde su uso permite separar linealmente dos clases, algo que en el espacio original no es posible.

En la etapa de experimentación de este trabajo se utilizaron SVMs con dos *kernels* distintos, por un lado el *kernel* lineal original, y por otro lado uno llamado RBF (radial basis function). Este último tiene un hiperparámetro γ, el cual define qué tan lejos tiene influencia cada punto de entrenamiento: valores bajos generan influencia en zonas más alejadas del punto, mientras que valores altos mantienen su influencia más cerca del punto. Puede ser visto como la inversa del radio de influencia de las muestras elegidas por el modelo como vectores soporte.

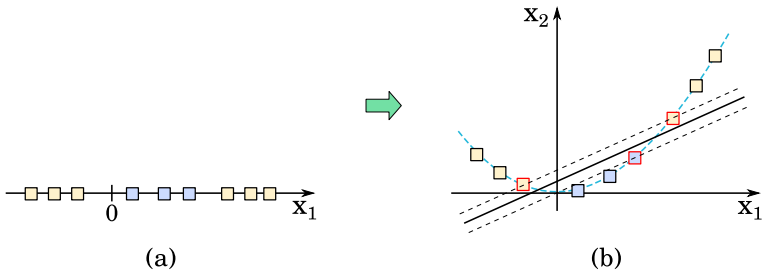


Figura 2.6: Problema resuelto utilizando un kernel. En el espacio original del problema no es posible separar los puntos con un hiperplano (a). Sin embargo, luego de aplicar un mapeo hacia un espacio bidimensional, utilizando el *kernel* $\varphi([x_1]) = [x_1, {x_1}^2]$, las clases pueden separarse con una recta (b). En rojo se destacan los vectores soporte.

2.7.1.2. Clasificación multiclase

Como en principio las SVMs sólo soportan clasificación para dos clases (una clase a cada lado del hiperplano), si se desea tener un clasificador multiclase es necesario combinar más de uno.

Los dos enfoques utilizados en este trabajo para lograrlo son los llamados uno-contra-uno (*one vs. one*) y uno-contra-el-resto (*one vs. the rest*):

- En **uno-contra-uno**, para un problema con k clases, se entrenan $k * (k-1)/2$ clasificadores binarios, es decir, uno por cada par de clases posibles, y cada clasificador debe aprender a distinguir entre sus dos clases asignadas. Al momento de realizar las predicciones, se aplica un esquema de votación.

- Para **uno-contra-el-resto**, la estrategia consiste en entrenar un solo clasificador binario por cada clase, con las muestras de esa clase como los ejemplos positivos y todas las otras muestras como los ejemplos negativos. La clase ganadora para una muestra resulta ser aquella cuyo clasificador binario asociado presenta la mayor distancia positiva entre su hiperplano y la muestra.

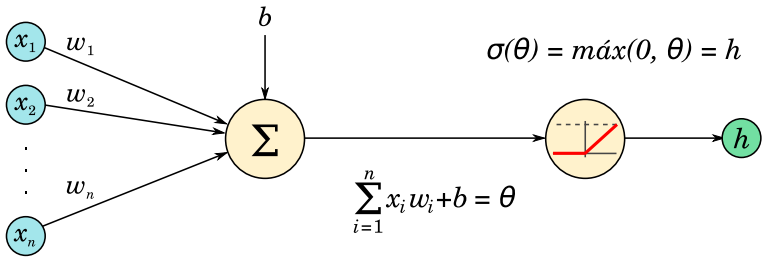


Figura 2.7: Representación gráfica de una neurona. En este ejemplo se utiliza la función de activación ReLU.

2.7.2. Redes neuronales

El término "red neuronal" tiene sus orígenes en intentos de encontrar representaciones matemáticas de procesamiento de la información en sistemas biológicos. Luego evolucionaron en valiosas herramientas utilizadas tanto para clasificación como regresión, ya que proveen un enfoque robusto para aprender funciones de valores reales, discretos y vectoriales a partir de un conjunto de ejemplos, en lo que hoy se conoce como *redes neuronales artificiales* (o simplemente redes neuronales). Se aplican de manera exitosa en numerosos campos del aprendizaje automatizado, como visión por computadora, traducción de texto, reconocimiento de voz, análisis financiero, entre otros.

El modelo más utilizado de redes neuronales es el de perceptrones multicapas, compuesto de un conjunto de unidades de procesamiento elementales, llamadas neuronas, conectadas convenientemente en capas.

2.7.2.1. Neuronas

Cada neurona toma como entrada una cantidad de valores reales (un vector) y calcula un único valor real como salida, a través de una combinación lineal de éstos y la aplicación de una función de activación. Esta salida representa su grado de activación, la cual a su vez suele ser la entrada de otras unidades conectadas a la misma. En la Figura 2.7 puede verse un ejemplo.

Una neurona o perceptrón toma el vector de entrada $\vec{x} = [x_1, x_2, ..., x_n] \in \mathbb{R}^n$ y produce una salida $v \in \mathbb{R}$. Llamaremos $\vec{w} = [w_1, w_2, ..., w_n]^T \in \mathbb{R}^n$ al vector de pesos que utilizará para hacer la combinación lineal de los valores de entrada y φ a la función de activación aplicada posteriormente. Además,

se agrega un parámetro b llamado *bias* de la neurona, el cual permite que el argumento de la función de activación no pase necesariamente por el origen de coordenadas [14]. Entonces nos queda que

$$v = \varphi\left(\sum_{i=1}^{n} x_i w_i + b\right).$$

Por otro lado, las funciones de activación pueden tener varias formas, las utilizadas en este trabajo son:

- **Sigmoide**:
$$\varphi(x) = \frac{1}{1+e^{-x}}$$
- **Tangente hiperbólica**:
$$\varphi(x) = \tanh(x)$$
- **ReLU** (*rectified linear unit*, o unidad lineal rectificada):
$$\varphi(x) = \text{máx}(0, x)$$
- **LReLU** (*leaky rectified linear unit*):
$$\varphi(x) = \begin{cases} \alpha \cdot x, & \text{si } x < 0 \\ x, & \text{si } x \geq 0 \end{cases}$$
Es similar a la anterior, pero con un gradiente distinto de 0 para entradas negativas, lo cual suele ayudar a la convergencia. En este trabajo se utiliza un valor de $\alpha = 0,2$.

En la Figura 2.8 se grafican las funciones de activación explicadas anteriormente.

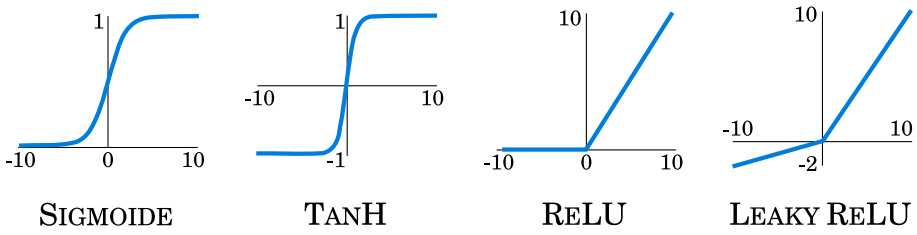


Figura 2.8: Funciones de activación.

2.7.2.2. Perceptrón multicapa

Al componer neuronas, agrupadas en capas, se forma entonces una red neuronal (ver Figura 2.9). Cada capa está formada por una serie de neuronas que no se encuentran conectadas entre sí, sino que cada una está conectada a la salida de una o más neuronas de la capa que la precede.

Conectando varias capas se crea una estructura unidireccional (*feedforward*) donde la salida de todas las capas, excepto de la última, se conecta a la capa siguiente.

Las entradas de la capa de entrada se conectan al vector de atributos del problema, mientras que las salidas de la capa de salida se consideran la salida de la red. Generalmente, en cada capa se utiliza la misma función de activación para todas las neuronas.

Las capas que no son la de entrada ni la de salida se denominan capas ocultas.

2.7.2.3. Capa softmax

Cuando se utilizan redes neuronales para resolver problemas multiclase, lo usual es definir una arquitectura cuya capa de salida posee una neurona por cada clase. Esta neurona idealmente se activará si y sólo si la muestra de entrada corresponde a dicha clase. Además, es deseable que al realizar la predicción para tal entrada la salida indique la probabilidad que tiene la misma de pertenecer a cada clase. Esta es precisamente la función que cumple la capa softmax, cuya salida en la j-ésima neurona para una entrada (de la capa, no de la red) $\mathbf{x}$ está dada por

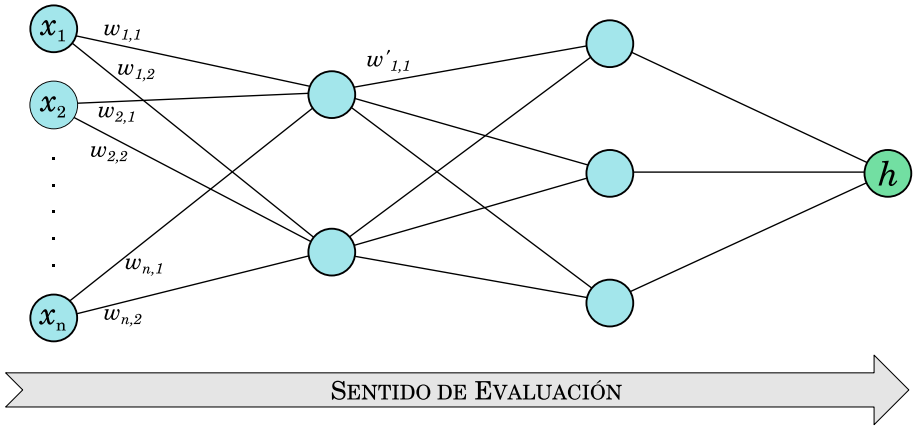


Figura 2.9: Ejemplo de una red neuronal.

$$\varphi(\mathbf{x})_j = \frac{e^{\mathbf{x}_j}}{\sum_{k=1}^{K} e^{\mathbf{x}_k}},$$

donde K es el número total de neuronas en la capa.

2.7.2.4. Descenso por gradiente

El método de descenso por gradiente es un algoritmo utilizado para minimizar una función, moviéndose iterativamente en la dirección del descenso más empinado. Es un método de primer orden, ya que sólo utiliza la primera derivada de la función que se desea minimizar.

Si tenemos una función de error $E(w)$ que nos indica cuán mal está la salida de una red dado un vector de parámetros w, podemos minimizarla por medio este método, como se muestra a continuación.

Para dicha función diferenciable, el vector de gradientes se compone de las derivadas parciales y se define como

$$\nabla_w E = \left[\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, ..., \frac{\partial E}{\partial w_d}\right]^T.$$

El procedimiento mencionado para minimizar E comienza a partir de un w aleatorio y en cada paso actualiza w en la dirección opuesta del gradiente, de la forma

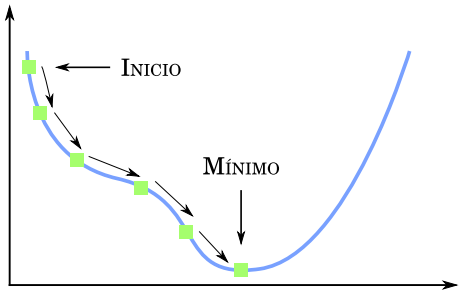


Figura 2.10: Ejemplo de descenso por gradiente para una función con entrada unidimensional.

$$w_i := w_i + \Delta w_i, \forall i,$$

donde

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}, \forall i$$

y $\eta > 0$ es el llamado *learning rate*, el cual determina qué tan grandes son los pasos que se dan en cada iteración del descenso por gradiente. Cuando se llega a un mínimo, la derivada es 0 y el procedimiento finaliza. Esto indica que el mismo encontró el mínimo más cercano, el cual puede ser un mínimo local, ya que no hay garantía de encontrar el mínimo global a menos que la función tenga solamente un mínimo. El uso de un buen valor de η es crítico, debido a que si es muy pequeño la convergencia puede ser muy lenta, y si es muy grande puede causar oscilaciones e incluso divergencia [10].

En la Figura 2.10 se observa un ejemplo muy simplificado de descenso por gradiente utilizando una función con entrada unidimensional.

2.7.2.5. Modificaciones del algoritmo de descenso por gradiente

Con el pasar de los años, han aparecido muchas variantes del algoritmo de descenso por gradiente. A continuación describiremos las que tienen impacto en este trabajo.

2.7.2.5.1. Descenso de gradiente estocástico

El descenso de gradiente estocástico (SGD) es una simplificación drástica. En lugar de calcular exactamente el gradiente de $E(w)$ (la función que incluye el error de todo el conjunto de datos), en cada iteración se estima el gradiente utilizando una sola instancia elegida aleatoriamente del conjunto de datos de entrenamiento [15].

La convergencia de este método ha sido estudiada en numerosas ocasiones. El teorema Robbins-Siegmund establece las bases para (bajo ciertas condiciones) casi asegurar la convergencia, la cual esencialmente se encuentra limitada por el ruido estocástico inducido por la elección aleatoria de una instancia en cada iteración, y no tanto por el valor inicial de los parámetros a ajustar ni por el número total de muestras. Estos resultados se pueden lograr inclusive en casos en donde la función no es diferenciable en todo su dominio [15].

2.7.2.5.2. SGD de minilote

El descenso de gradiente estocástico de minilote (SGD de minilote) consiste en una combinación entre SGD y la iteración utilizando el lote completo. Generalmente, el tamaño de un minilote oscila entre 10 y 1000 instancias, elegidos de manera aleatoria. Este método reduce la cantidad de ruido introducida en el SGD y sigue siendo más eficiente que utilizar el lote completo [16].

2.7.2.5.3. Momento lineal (*momentum*)

Supongamos que se está entrenando una red y w_i corresponde a algún peso en alguna capa de esa red. En cada actualización del parámetro, los sucesivos valores Δw_i^t podrían ser tan diferentes que produzcan grandes oscilaciones, ralentizando la convergencia (la variable t corresponde al índice en el eje temporal del entrenamiento).

La idea del momento lineal es tomar un promedio de movimiento de los gradientes ponderado exponencialmente en el tiempo, incorporando actualizaciones anteriores a la actual [10]. Puede considerarse como un momento generado por las mismas, análogo al momento definido en el campo de la física [11]. Su fórmula está dada por

$$\Delta w_i^t = -\eta \frac{\partial E^t}{\partial w_i} + \alpha \Delta w_i^{t-1}.$$

En general, suele tomarse α entre 0,5 y 1,0. Esta modificación del método original es especialmente útil cuando se utiliza SGD, ya que como resultado se obtiene el efecto de promediar y suavizar la trayectoria durante la convergencia [10]. Otro objetivo consiste en disminuir las probabilidades de que el aprendizaje se atasque en mínimos locales, lo que impediría encontrar el mínimo global [11].

Presenta la desventaja de que se deben almacenar valores previos de Δw_i, lo cual requiere mayor cantidad de memoria.

2.7.2.5.4. *Learning rates* adaptables por parámetro

Otras de las modificaciones que se suelen realizar al método de descenso por gradiente consisten en calcular y utilizar diferentes *learning rates* adaptables en el tiempo para cada uno de los parámetros que se están ajustando. En este trabajo utilizamos el algoritmo Adam para entrenar las redes, por lo que comentaremos muy brevemente de qué se tratan dos algoritmos que lo preceden y finalizaremos explicando Adam.

AdaGrad Este método [17] modifica los *learning rates* de los parámetros en base a su frecuencia de actualización, utilizando valores grandes para parámetros con actualizaciones poco frecuentes y valores pequeños (disminuyendo el tamaño de los pasos) para parámetros con actualizaciones frecuentes. En términos más simples, cuantas más actualizaciones recibe un parámetro, más pequeñas son éstas. Por esta razón es útil cuando se trabaja con datos dispersos. Conserva el hiperparámetro *learning rate* global (compartido por todos los parámetros), el cual se utiliza junto al *learning rate* dinámico de cada parámetro. Su mayor debilidad es que debido a su formulación (en la cual acumula gradientes anteriores como términos de una suma que se encuentra en un denominador) al avanzar en la cantidad de iteraciones los *learning rates* dinámicos tienden a reducirse hasta volverse infinitesimalmente pequeños al punto en que el algoritmo no es capaz de adquirir nuevo conocimiento [18]. Su nombre proviene de *adaptive gradient* (gradiente adaptable).

RMSprop Este otro algoritmo, llamado así por las siglas de *root mean*

square (media cuadrática), fue presentado por primera vez en una clase[1] de Geoff Hinton. Ataca el problema de los *learning rates* que decrecen indefinidamente en Adagrad. Consiste en dividir el *learning rate* por un promedio ponderado de los cuadrados de los gradientes que decae exponencialmente [18].

Adam Es otro método que calcula *learning rates* adaptables para cada parámetro [19]. Además de almacenar un promedio ponderado que decae exponencialmente de los cuadrados de gradientes anteriores (como RMSprop), Adam también mantiene un promedio ponderado de los gradientes anteriores que decae exponencialmente, similar al momento lineal, pero utilizado para calcular *learning rates* [18]. Es nombrado así por *adaptive moment estimation*.

2.7.2.6. Entrenamiento con *backpropagation*

El entrenamiento de la red consiste en obtener valores para los w, es decir, fijar los pesos utilizados en los productos que realizan los perceptrones.

El método más utilizado para esto se llama retropropagación (en inglés, *backpropagation*). Tiene como objetivo minimizar el error de la red, utilizando el método de descenso por gradiente explicado anteriormente. Este algoritmo puede aplicarse para cualquier red neuronal de tamaño arbitrario.

En cada iteración del entrenamiento, backpropagation hace una propagación común (hacia adelante) a través de la red y luego actualiza los pesos de cada neurona, desde la capa de salida hasta la de entrada, realizando un paso en la dirección que indique el algoritmo del descenso por gradiente para la función de error asociada.

2.7.3. Redes neuronales profundas

En las redes neuronales profundas, se aprovecha el efecto producido al apilar muchas capas de componentes adaptativos no lineales. Esto permite representar una amplia familia de funciones de manera más compacta que utilizando arquitecturas poco profundas, como redes neuronales de una o dos capas ocultas.

[1] `http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf`

Pese a las ventajas que poseen este tipo de redes, no fue hasta el año 2006, en que Hinton y otros propusieron un método para entrenarlas de manera satisfactoria [20]. De manera muy simplificada, consistía en preentrenar cada par de capas con una heurística *greedy* (voraz), lo cual hace que el entrenamiento de la red profunda utilizando descenso por gradiente comience con pesos inicializados en una región cercana al óptimo buscado.

En las redes profundas, cada nivel aprende a transformar sus datos de entrada en una representación ligeramente más abstracta y compuesta. Por ejemplo, en una aplicación de reconocimiento de imágenes, la entrada puede ser una matriz de píxeles, la primera capa puede abstraer los píxeles y codificar bordes, la segunda capa puede componer y codificar conjuntos de bordes; la tercera capa puede codificar una nariz y ojos, y la cuarta capa puede reconocer que la imagen contiene una cara.

Durante este proceso de aprendizaje, la red puede aprender qué características asignar a qué nivel de abstracción de manera óptima por sí sola. Por supuesto, esto no elimina el proceso de ajuste de hiperparámetros, por ejemplo, al variar la cantidad de capas y sus tamaños se pueden obtener diferentes grados de abstracción [21].

Las redes profundas son utilizadas para resolver una gran variedad de problemas de diversas áreas, tales como análisis de imágenes médicas [22, 23, 24], conducción autónoma [25, 26], traducción automática de textos [27], reconocimiento [28, 29] y procesamiento [30, 31] de imágenes, entre otras.

2.7.3.1. Redes convolucionales

Estas redes fueron descriptas por LeCun y otros en 1989 al aplicarlas en el reconocimiento de códigos postales manuscritos [32] y posteriormente en 1998 (donde nuevamente las utiliza para reconocimiento de texto manuscrito) [33].

En las redes neuronales tradicionales, la red recibe una entrada (un vector) y lo transforma a través de una serie de capas ocultas. Cada capa oculta está compuesta por un conjunto de neuronas, donde cada neurona está conectada a todas las de la capa anterior y las neuronas de una capa funcionan de manera completamente independiente y no comparten ninguna conexión. La última capa es la llamada capa de salida y, si se trata de clasificación, representa los puntajes de cada clase.

Por otro lado, en las redes convolucionales [14, 9] estas disposiciones son diferentes. Las neuronas de una capa no están siempre conectadas con todas

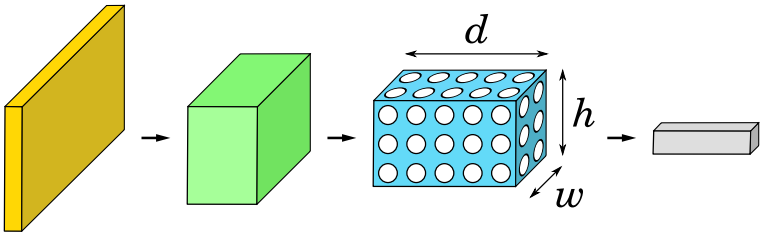


Figura 2.11: Diagrama de una red neuronal convolucional. En la tercera capa se observan sus neuronas, dispuestas en tres dimensiones (ancho, alto y profundidad). Fuente: A. Karpathy (2016) [34].

las de la capa anterior y entre las neuronas de una misma capa no hay completa independencia. Cada capa es un arreglo de tres dimensiones, de tamaño $h \times w \times d$, donde h y w son las dimensiones espaciales y d es la dimensión de canales (ver Figura 2.11).

En caso de que la red tome como entrada una sola imagen, la primera capa corresponde a la misma, de tamaño $h \times w$ píxeles y d canales de colores. En este trabajo, donde la entrada está compuesta por una sucesión de imágenes, la primera capa mantiene el tamaño $h \times w$ píxeles, correspondiente a las dimensiones de las imágenes, y d indica la cantidad total de canales (estará dada por el producto entre la cantidad de imágenes o frames que componen la secuencia y la cantidad de canales por cada uno de ellos). Las ubicaciones en las capas superiores corresponden a las ubicaciones en la imagen que se encuentran conectadas a ellas a través de un camino[2].

Cada capa de una red convolucional transforma el volumen de entrada 3D a un volumen de salida 3D de activaciones de neuronas, utilizando alguna función que puede o no tener parámetros.

Una característica central de las redes convolucionales es que presentan invarianza traslacional, es decir que pueden reconocer un determinado objeto aunque éste se encuentre en una ubicación diferente de la imagen. Esto lo logran gracias a que sus componentes básicos (más adelante se explicarán los diferentes tipos de capas) operan en regiones locales de la entrada, dependiendo sólo de las coordenadas espaciales relativas, y no de la ubicación absoluta en la que se encuentran aplicados [35].

[2]Viendo a la red neuronal como un grafo dirigido, donde las neuronas son los vértices, y las conexiones entre ellas son las aristas (con el mismo sentido en que fluye la información).

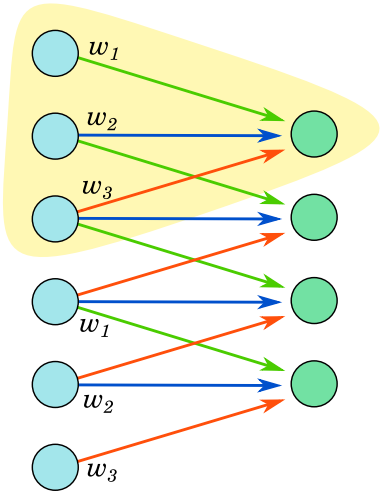


Figura 2.12: Unidades en una red convolucional. Todas las unidades de la derecha comparten los mismos parámetros w_i. Cada una computa la misma función, pero con diferente ventana de entrada (analiza una ubicación diferente de la imagen). El color de la flecha indica el peso de esa conexión. Fuente: R. Girshick (2014) [37].

Al emplear la misma función en diferentes ubicaciones de la imagen (pesos "atados") se reduce significativamente la cantidad de parámetros que deben ser aprendidos por la red, lo que hace posible entrenarla con menos ejemplos que si diferentes funciones fueran aprendidas en ubicaciones distintas (pesos "desatados") [36]. La Figura 2.12 ilustra dicha característica, para una capa con cuatro neuronas, las cuales comparten tres pesos.

Cuando se construyen redes convolucionales, pueden utilizarse diferentes tipos de capas, las cuales transforman los valores de entrada en valores de salida de diferentes formas, cumpliendo funciones específicas. A continuación se explican los tipos de capas utilizados en este trabajo.

Capa convolucional

Es el tipo de capa que le da nombre a este tipo de redes. Sus parámetros consisten en un conjunto de filtros entrenables. Cada filtro es pequeño espacialmente, es decir, con respecto al ancho y alto de la entrada, pero se extiende a través de toda la profundidad del volumen de entrada. Durante la pasada hacia adelante (*forward pass*), cada filtro se desplaza (o, más precisamente, convoluciona) a lo ancho y alto del volumen de entrada y computa productos escalares entre los pesos del filtro y la entrada para cada posición. En la Figura 2.13 se pueden observar neuronas pertenecientes a diferentes filtros, pero ubicadas espacialmente en el mismo lugar, por lo que observan la misma superficie de entrada, aunque computan diferentes funciones.

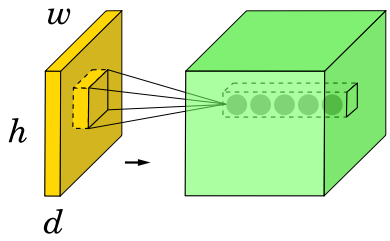


Figura 2.13: Campo receptivo de las neuronas en las redes convolucionales. Cada neurona se encuentra conectada solamente a una región local (espacialmente) del volumen de entrada. Sin embargo, se encuentra conectada con toda la profundidad del mismo, es decir que tiene acceso a todos los mapas de activación del volumen que la precede (o a todos los canales de la imagen, si se trata de la entrada de la red). Fuente: A. Karpathy (2016) [34].

A medida que el filtro es desplazado por el ancho y alto del volumen de entrada, produce un mapa de activación de dos dimensiones que da las respuestas de ese filtro en cada posición espacial. Intuitivamente, la red aprenderá filtros que se activan cuando detectan algún tipo de característica visual, como por ejemplo algún borde con determinada orientación, o una mancha de un color en particular en la primera capa, o eventualmente estructuras más complejas como forma de panales de abejas o patrones circulares en capas superiores de la red. Entonces, obtendríamos un conjunto de filtros en cada capa convolucional, donde cada uno de ellos produce un mapa de activación bidimensional separado. Podemos apilar estos mapas de activación en el sentido de la dimensión de profundidad y así producir el volumen de salida.

Además del tamaño del filtro, las capas de este tipo presentan tres parámetros que deben ser fijados al momento de definir la arquitectura de la red:

- **Profundidad**: indica la cantidad de filtros que contiene la capa. Cada uno de estos filtros es independiente y buscará distintas características en los datos de entrada.

- ***Stride*** (o paso): establece cómo se mueven los filtros a lo ancho y alto de la imagen. Un valor de 1 indica que los filtros se mueven de a un píxel a la vez, un valor de 2 indica que el filtro da un salto de dos píxeles en lugar de uno entre una posición y la siguiente, y así sucesivamente.

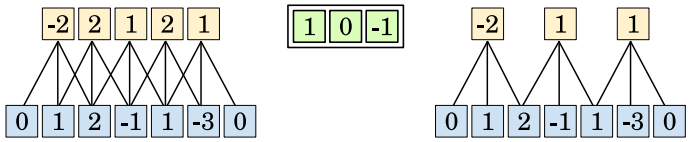


Figura 2.14: Ejemplos de tamaño del filtro, *stride* y *zero-padding* generando mapas de activación de diferentes tamaños. Fuente: A. Karpathy (2016) [34].

A medida que aumenta este valor, disminuye espacialmente el volumen de salida de la capa.

- ***Zero-padding***: en algunas ocasiones, es conveniente completar el volumen de entrada con ceros alrededor del borde del mismo, incrementando sus dimensiones espaciales. Esto tiene como finalidad controlar arbitrariamente el tamaño espacial del volumen de salida. En el caso más común, se utiliza para evitar que en cada capa posterior se reduzca de forma indeseada el tamaño del volumen de salida, algo que ocurre debido a la naturaleza de la convolución si no se agrega *padding*. Este parámetro define el grosor de este nuevo borde conformado por ceros.

Conociendo los valores de estos hiperparámetros y el tamaño del volumen de entrada es posible calcular el volumen de salida. Si el volumen de entrada tiene dimensiones $w_i \times h_i \times d_i$, los filtros son n (profundidad) y tienen un tamaño de $w_f \times h_f$, el *stride* es s y el *zero-padding* es p, entonces el tamaño del volumen de salida será $w_o \times h_o \times n$, donde $w_o = (w_i - w_f + 2p)/s + 1$ y $h_o = (h_i - h_f + 2p)/s + 1$. En la Figura 2.14 hay dos ejemplos con diferentes *strides*, simplificado a una sola dimensión en lugar de dos.

En este ejemplo, los valores de los hiperparámetros son $w_i = 5$, $w_f = 3$ y $p = 1$. En el caso de la izquierda, se tiene que $s = 1$, por lo que $w_o = (w_i - w_f + 2p)/s + 1 = (5 - 3 + 2 * 1)/1 + 1 = 5$, como se ve en la capa superior. En el caso de la derecha, se trabaja con $s = 2$, por lo que $w_o = (w_i - w_f + 2p)/s + 1 = (5 - 3 + 2 * 1)/2 + 1 = 3$. En el centro se muestran los tres pesos utilizados en ambos ejemplos (se utilizan tres pesos porque es el tamaño del filtro, todas las neuronas comparten estos mismos pesos).

Capa convolucional traspuesta

Este tipo de capa, también conocida como capa deconvolucional o de *strides* fraccionales, es utilizada en la construcción de las redes adversarias

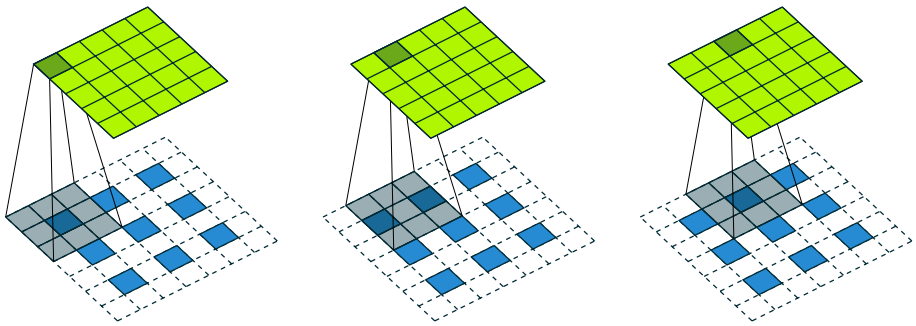


Figura 2.15: Convolución traspuesta de una convolución con un filtro de 3×3 sobre una entrada de 5×5 con un *zero-padding* de 1, usando $stride = 2$. Es equivalente a convolucionar con un filtro de 3×3 sobre una entrada de 3×3 (con 1 cero insertado entre cada elemento de la entrada) con un *zero-padding* de 1, usando $stride = 1$. En los mapas inferiores, las activaciones con bordes en línea de puntos corresponden a los ceros añadidos y las activaciones oscuras corresponden a los valores originales, provenientes de una capa anterior. Fuente: V. Dumoulin y F. Visin (2016) [38].

generativas, más específicamente en el generador (el cual explicaremos más adelante).

Esta capa ejecuta una propagación hacia atrás de la correspondiente convolución no-traspuesta. Su característica distintiva de la capa convolucional común es que genera una salida espacialmente mayor que su entrada, por lo que es particularmente útil para generar imágenes más grandes a partir de otras más pequeñas. Se puede pensar como si se expandiera la entrada añadiendo ceros entre elementos adyacentes de la misma (y el *zero-padding* necesario) y luego se ejecutara una capa convolucional sobre esta nueva entrada (ver Figura 2.15). De esta forma, este procedimiento es capaz de generar un volumen de salida con dimensiones mayores a las de entrada.

Capa de *pooling* (agregación)

A esta capa se la utiliza principalmente para reducir de manera progresiva el tamaño espacial de las salidas de las capas, reduciendo así la cantidad de parámetros y cálculos en la red y, por lo tanto, también controlar el sobreajuste [34]. Este tipo de capa opera de forma independiente en cada nivel de profundidad de la entrada y la redimensiona espacialmente, usando

alguna operación de agregación (como puede ser *máx*). Habitualmente se utilizan filtros de tamaño 2×2, con un *stride* de 2, haciendo que se reduzca a la mitad el ancho y alto del volumen de entrada, descartando el 75 % de las activaciones. En este caso (y suponiendo que se utiliza *máx*), en cada operación *máx*, la capa está tomando el máximo de cuatro números (una región de 2×2 con profundidad 1, ya que actúa independientemente en cada nivel de profundidad). La profundidad del volumen siempre se mantiene para este tipo de capa.

Otra configuración, ligeramente menos común que la ya mencionada, consiste en utilizar un tamaño de filtro 3×3, con un *stride* de 2 [34]. Estos valores hacen que dos neuronas contiguas del mapa de activación tengan algunas entradas en común. No suelen utilizarse tamaños de filtro mayores que 3, ya que pueden resultar muy destructivos al descartar demasiados valores, lo que generalmente conduce a un peor desempeño. Una excepción a esto se da en Lin, Chen y Yan (2013) [39], en donde utilizan una capa de *pooling* con tamaño de filtro igual a su propia entrada (es decir que el *stride* no se utiliza) junto con la función promedio, conectando cada mapa de activación a sólo una de las neuronas de salida (ubicada en una capa softmax), prescindiendo así del uso de capas *fully connected*.

Otra función que puede utilizarse en lugar de la función *máx* es la función *promedio*, o incluso normas vectoriales, pero la más usual es la función *máx*, debido a que ha demostrado funcionar mejor en la práctica [34]. Otro punto importante es que las capas de *pooling* no tienen ningún parámetro que deba entrenarse, ya que realizan el cálculo de una función fija.

Capa de normalización

Las capas de normalización, como su nombre lo indica, normalizan los datos de alguna manera. Como hay diferentes tipos de capas en esta categoría, explicaremos la única capa de normalización utilizada en este trabajo, llamada capa de normalización por lotes (*batch normalization*).

La función de esta capa es la de transformar los datos para que tengan una media de 0 y una varianza de 1. Para ello, durante el entrenamiento, en cada batch que procesa ajusta las estadísticas promedio de todos los datos de entrenamiento. Luego, durante la ejecución de la red utiliza estos datos para realizar la transformación adecuada.

Capa *fully-connected* (completamente conectada)

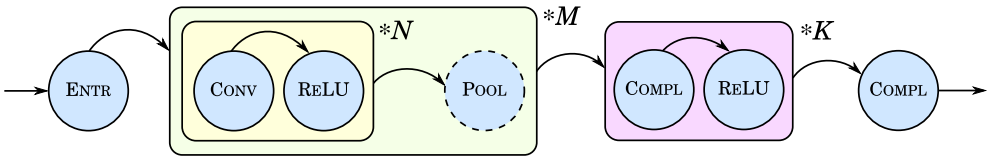


Figura 2.16: Forma genérica más común de una red convolucional. El símbolo $*$ indica repetición, las constantes $N, M, K \in \mathbb{N}_0$ son elegidas al definir la arquitectura y por lo general $N \leq 3$ y $K < 3$. La capa de *pooling* es opcional.

Las capas de este tipo son las utilizadas en las redes neuronales convencionales. Las neuronas presentes en este tipo de capa se encuentran conectadas a la salida de todas las neuronas de la capa anterior.

2.7.3.1.1. Arquitecturas de redes convolucionales

Para construir una red convolucional deben combinarse los tipos de capas mencionados anteriormente. Además, consideraremos la función de activación ReLU como una capa más, que aplica no-linealidad a cada neurona de la capa anterior.

La forma más común de arquitectura de redes convolucionales consiste en apilar algunas capas convolucionales con capas ReLU, seguidas de capas de *pooling*, y repetir este patrón hasta que la imagen haya sido reducida espacialmente a un tamaño pequeño. En algún punto, es usual transicionar a capas completamente conectadas hasta una última capa, cuya salida es la de la red y contiene el puntaje asignado a cada clase. En la Figura 2.16 se observa la anatomía genérica más común para este tipo de redes.

2.7.4. Redes adversarias generativas

Las redes adversarias generativas (en inglés, generative adversarial networks o GANs), como se propone en Goodfellow et al. [1], son una clase de métodos no supervisados en los cuales se utilizan dos redes neuronales que compiten por lograr objetivos opuestos.

Por un lado se tiene una red llamada generador, la cual produce muestras con la distribución de los datos tomando como entrada vectores de ruido aleatorio con distribución uniforme. Esta red es entrenada para engañar al discriminador, haciendo que acepte sus salidas como si fueran muestras reales

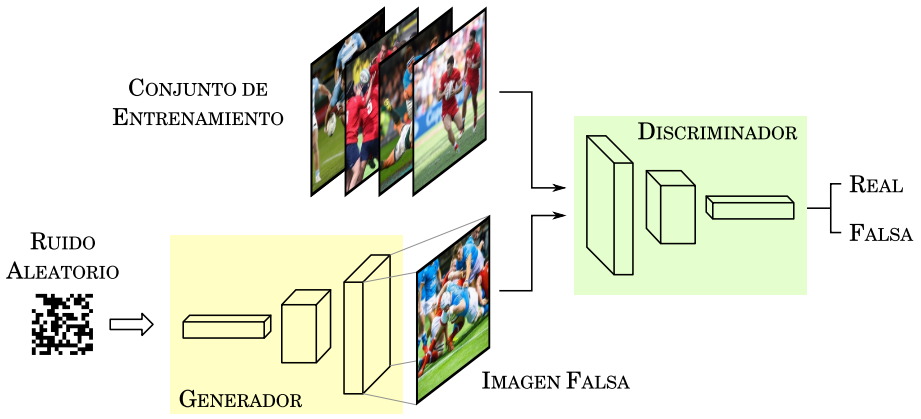


Figura 2.17: Anatomía de una red GAN. Fuente: S. Thalles (2017) [41].

[40]. Para ello, debe generar muestras con características tan similares a las muestras reales como le sea posible.

Por otro lado, el discriminador es una red cuyo objetivo es distinguir entre las muestras sintetizadas por el generador y las muestras reales. Toma en su entrada una imagen que puede provenir de los datos reales o del generador. Su última capa está compuesta por una sola unidad, la cual tomará un valor u otro dependiendo de que la imagen de entrada sea considerada por la red como una muestra de los datos reales o una imagen sintetizada por el generador, es decir que es un clasificador binario.

En la Figura 2.17 pueden observarse las dos redes mencionadas que componen una red GAN, junto con la conexión que existe entre ellas y los datos de entrada.

El entrenamiento de las redes GAN consiste en realizar un gran número de iteraciones, donde cada una efectúa actualizaciones en los pesos de ambas subredes, en dos etapas diferentes.

En la primera etapa se modifican los parámetros del discriminador, con el objetivo de que éste sea capaz de clasificar correctamente las entradas en "real" o "falsa". Para ello, se utilizan muestras reales del conjunto de datos no etiquetados, y se define como valor objetivo para cada muestra la clase "real". También se utilizan muestras producidas por el generador a partir de vectores de ruido aleatorio. En este caso a dichas muestras se les asigna el valor objetivo de la clase "falsa". En esta etapa sólo se modifica el discriminador.

En la segunda etapa de la iteración se ajustan los pesos del generador. Para ello, sólo se utilizan como entrada conjuntos de vectores de ruido aleatorio uniformemente distribuido. En este caso, se define como valor objetivo de la red (en la salida del discriminador) la etiqueta "real" para todas las muestras. Esta es la clave para lograr que el generador modifique sus pesos para producir salidas que engañen al discriminador, con el efecto secundario buscado de aprender a generar imágenes muy parecidas a las del conjunto de imágenes reales.

Un punto muy interesante a destacar es que el generador nunca "ve" (procesa) las imágenes que intenta imitar. Solamente tiene la información que puede extraer de los pesos aprendidos por el discriminador (el cual sí tiene acceso a las imágenes).

A medida que se va entrenando el generador, el discriminador debe observar detalles de cada vez más alto nivel, mejorando su representación de la entrada. Al finalizar el entrenamiento de forma exitosa, y suponiendo que se llega al óptimo teórico, el generador produce imágenes indistinguibles de las reales, por lo que el discriminador ya no puede aprender nada al no existir información para extraer que diferencie las muestras reales de las falsas. De todas formas, el discriminador habrá recibido un entrenamiento suficiente para poder capturar y abstraer la información más importante de las imágenes de entrada.

Capítulo 3

Experimentación

El problema abordado en este trabajo consiste en clasificar jugadas de rugby, donde cada una pertenece a una de tres clases posibles: '*juego*', '*line*' o '*scrum*'. Este problema presenta algunas dificultades comunes a la mayoría de los problemas de visión computacional, junto a algunas complicaciones adicionales particulares del mismo. Para poder clasificar correctamente las jugadas, los métodos utilizados deben ser capaces de ignorar elementos irrelevantes presentes en los videos, como objetos fuera del campo de juego (por ejemplo: árboles, automóviles, postes de alumbrado, tribunas, espectadores), condiciones climáticas, sombras, movimientos y ángulos de las cámaras, niveles de zoom, tipos de césped o irregularidades en el campo de juego. Por otro lado, deben utilizar adecuadamente los movimientos de los jugadores dentro del campo, que son los que definen la clase que se debe asignar a cada jugada.

A continuación presentamos un resumen de los experimentos que se describen en este capítulo. Más adelante se describen los datasets utilizados, junto con las transformaciones aplicadas a los mismos para adecuarlos a los algoritmos que se emplearán y, finalmente, los experimentos propiamente dichos.

Primero se intentará mostrar que con DCGAN es posible obtener en el discriminador una representación de los datos donde se capture en las direcciones principales variaciones con un mayor nivel de abstracción que en los datos originales.

Para ello, se aplica PCA a un conjunto de imágenes provenientes de un dataset no etiquetado, compuesto por segmentos de partidos de rugby. Cada píxel de las imágenes corresponde a una variable de entrada o atributo. Luego

se generan histogramas utilizando los diez componentes principales hallados por PCA y se toman imágenes aleatoriamente de cada bin perteneciente a algún histograma. Se realiza una inspección visual de las mismas para analizar qué significado tiene cada histograma (asociado a un componente). No se espera que este enfoque obtenga resultados interesantes, ya que al analizar los píxeles desde un nivel tan bajo, es difícil que se logre un nivel elevado de abstracción.

Siguiendo con el uso del dataset no etiquetado, se construye y entrena una red DCGAN. Se analizan visualmente las imágenes producidas por el generador y se toman las salidas producidas por la penúltima capa del discriminador. Estas salidas consisten en una representación o codificación resumida del conjunto de datos y se utilizan más adelante.

A los vectores representación anteriormente mencionados se les aplica PCA para, al igual que se hizo antes, analizar mediante sus histogramas los componentes principales hallados. Se espera que este procedimiento genere particiones con mayor nivel de abstracción que las del primer paso.

Posteriormente se investiga el uso de esta representación más abstracta en el problema de clasificación planteado. Con este fin se comienza a trabajar con videos de jugadas de rugby etiquetados. De la superficie de cada uno se toman nueve recortes, abarcando gran parte de ésta. Estos recortes se repiten a lo largo de la línea temporal del video, generando así una gran cantidad de ventanas o parches que barren la superficie y el tiempo de cada video. A esas ventanas se les aplica el discriminador entrenado previamente. Los vectores obtenidos son utilizados como entrada para entrenar algunas SVMs con diferentes hiperparámetros, con el objetivo de aprender a clasificar las ventanas correctamente.

Como método alternativo para lograr la clasificación de estos videos, se construye y entrena una red convolucional. La misma posee (luego de las capas típicas de redes convolucionales) una capa *fully connected* seguida de una capa softmax. En este caso, las ventanas ocupan la totalidad de la superficie del video, por lo que las mismas sólo limitan a las muestras temporalmente.

A esta red convolucional se le asignan como pesos iniciales los aprendidos durante el entrenamiento de la red GAN del comienzo. Esto es posible debido a que se utiliza una arquitectura muy similar en el discriminador de la red GAN y en la red convolucional (filtros con los mismos tamaños). Se prosigue con el entrenamiento de estos pesos, ahora de manera supervisada.

A diferencia del experimento con las SVMs, en este caso se hace un *fine-tuning* de la representación obtenida de manera no supervisada con GAN.

Desde otro punto de vista, puede considerarse a esta red como el mismo discriminador, al cual se le acopla una capa *fully connected* junto con una capa softmax. Se espera que la red inicializada con estos pesos logre un mejor desempeño que la red inicializada con pesos aleatorios.

Al construir los datasets derivados de los dos iniciales se consideran diferentes alternativas de tamaños y cantidad de frames por cada muestra. Respecto a los tamaños, se utilizan tres campos visuales diferentes, algo muy similar a realizar zoom a la imagen en mayor o menor medida, manteniendo el tamaño en píxeles de la imagen resultante. En cuanto a la cantidad de frames, los datasets generados tienen versiones con 1, 3, 5 y 10 frames. Todo esto se encuentra más detallado en la sección siguiente.

Finalmente, como hasta aquí se trabajó siempre con recortes limitados por hasta 10 frames de duración, se analizan algunas alternativas simples para utilizar los resultados de clasificación de estos segmentos individuales en predicciones para los videos completos del dataset etiquetado.

3.1. Datasets

En esta sección describiremos los datasets iniciales, así como las transformaciones que se les aplican para utilizarlos en los experimentos a lo largo del trabajo. Comenzaremos describiendo un dataset que contiene una gran cantidad de imágenes sin etiquetar, que será empleado en las etapas de aprendizaje no supervisado, y luego otro que contiene una menor cantidad de imágenes, esta vez etiquetadas, utilizado en las etapas de aprendizaje supervisado.

Ambos datasets están formados por videos con jugadas de rugby. Estos videos presentan el tipo de dificultades descriptas al comienzo de este capítulo, lo que los hace adecuados para aprender la clase de problemas con los que nos interesa trabajar. Estos videos fueron provistos por el Grupo de Investigación de Aprendizaje Automatizado y Aplicaciones del CIFASIS (CONICET), y contienen grabaciones de cámaras personales y emisiones televisivas de partidos de rugby o fragmentos de éstos.

3.1.1. Rugby no etiquetado

El primero de los conjuntos de videos contiene partidos de rugby completos sin ninguna metadata ni información sobre lo que está aconteciendo,

por lo que los modelos entrenados con este dataset no tendrán más que los frames de video como entrada.

Este tipo de datasets son los más fáciles de obtener, ya que no es necesario realizar ninguna tarea manual de etiquetado, simplemente pueden tomarse de fuentes habituales de videos, por ejemplo sitios web de *streaming* de videos o emisiones televisivas. Esto hace que al momento de entrenar estos modelos pueda recopilarse mucha información de entrada fácilmente. Algunas características de este conjunto de videos son:

Nº de videos	**Duración promedio**	**Duración total**
282	74, 2 min.	348, 8 hs.

Cuadro 3.1: Estadísticas del dataset de videos no etiquetados.

3.1.2. Rugby etiquetado

El segundo grupo se compone de videos cortos, donde cada uno está formado por jugadas tomadas de algún partido de rugby. Cada una de estas jugadas se encuentra encuadrada en una de tres clases y cada video puede contener jugadas de una sola clase. Las clases son:

'Line' El *line-out* es una forma de reiniciar el juego después de que la pelota o un jugador que la porta cruza la línea de touch. Consiste en un saque de banda, que debe arrojarse recto y superando la línea ubicada a cinco metros campo adentro del touch entre dos hileras de jugadores, una de cada equipo y separadas por una distancia de un metro. Para obtener la posesión, cualquier jugador en el *line-out* puede saltar por la pelota, pudiendo ser impulsado y sostenido por dos compañeros [42].

'Scrum' El *scrum* es un reinicio del juego que se realiza después de una infracción menor. Se forma mediante el enfrentamiento de dos bloques (llamados packs) de jugadores enlazados, correspondiente cada uno a un equipo. Cuando se va a realizar un *scrum* cada uno de los equipos forma un bloque con sus ocho delanteros, siguiendo una estructura de tres líneas. Una vez formados, ambos packs deben agacharse y ubicarse uno frente al otro, para tomarse y esperar la indicación del árbitro para empezar a empujar con el fin de avanzar sobre el otro y obtener el balón (sin tocarlo con la mano), que ha sido lanzado en medio de ellos [42].

'Juego' Esta clase contiene el resto de las jugadas, es decir fragmentos de partidos que no contienen *lines* ni *scrums*.

Además, para cada uno de estos videos se conocen los nombres de los dos equipos que están participando. Esto es muy importante para que al momento de armar los datasets no se coloque un fragmento de un partido en el dataset de entrenamiento y otro fragmento del mismo partido en el dataset de prueba. Si eso ocurriese, no podría confiarse en la capacidad de generalización (ver 2.2) de los modelos producidos, ya que el dataset de prueba se vería contaminado por muestras del dataset de entrenamiento, haciendo que no sean independientes entre sí.

En la Figura 3.1 se pueden ver algunos ejemplos de los videos que componen este grupo, indicando la clase a la que pertenecen. A continuación, se muestran algunas características del conjunto de videos etiquetados, desagregado por clase:

Clase	**Nº de videos**	**Duración promedio**	**Duración total**
Line	57	$3,1$ mins.	$173,9$ mins.
Scrum	25	$3,4$ mins.	$84,1$ mins.
Juego	16	$1,6$ mins.	$25,0$ mins.

Cuadro 3.2: Estadísticas del dataset de videos etiquetados.

3.1.3. Preprocesamiento de los datos

Como inicialmente se tienen grupos de videos, estos datos no son aptos para trabajar en bruto con redes neuronales ni otros algoritmos utilizados en este trabajo. Por lo tanto, se le aplicarán ciertos preprocesamientos, los cuales incluyen realizar recortes de la superficie de los videos, redimensionarlos y tomar sólo cierta cantidad de frames, dando lugar a lo que llamamos ventanas.

3.1.3.1. Campo visual de ventanas

Un parámetro considerado de enorme importancia en este trabajo es el tamaño del campo visual utilizado durante el entrenamiento de las GANs, por eso es necesario explicarlo antes de detallar los pasos para generar los datasets derivados de los grupos de videos.



Figura 3.1: Selección de imágenes provenientes de los videos del dataset de rugby etiquetado. Las clases son '*juego*', '*line*' y '*scrum*', para la primera, segunda y tercera columna, respectivamente.

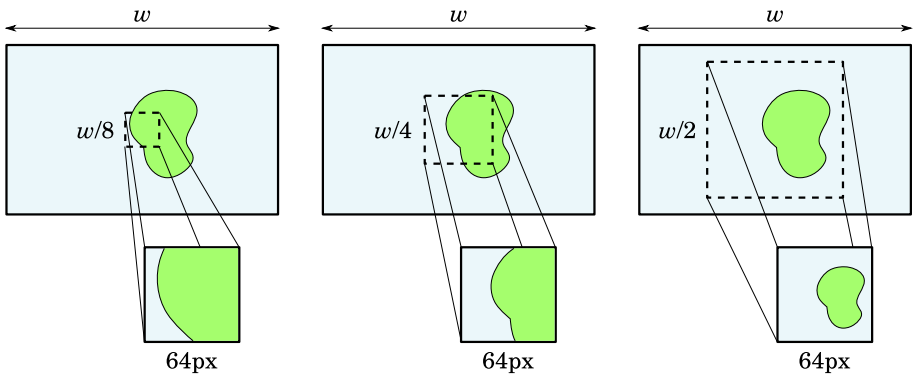


Figura 3.2: Campos visuales CE (Cerca), ME (Medio) y LE (Lejos). Sus tamaños finales son idénticos, producto del escalado que se aplica luego de cada recorte (para los datasets U y L1).

Utilizaremos tres tamaños de ventana o campos visuales, todos ellos de forma cuadrada, cada uno definido de acuerdo a la relación entre su tamaño y el ancho del video.

Sea w el ancho del video del que se extraerán las ventanas, entonces los campos visuales, junto con sus respectivas dimensiones son:

- **CE (Cerca):** $w/8 \times w/8$, o $12,5\,\%$ del ancho del video
- **ME (Medio):** $w/4 \times w/4$, o $25\,\%$ del ancho del video
- **LE (Lejos):** $w/2 \times w/2$, o $50\,\%$ del ancho del video

Pese a que cada campo visual tiene un tamaño diferente al momento de realizar el recorte sobre el video, cuando se construyen algunos de los datasets (específicamente U y L1, que serán explicados más adelante), las ventanas son redimensionadas a 64 px. × 64 px., para que puedan ser utilizadas como entrada de las redes neuronales que procesan dichos datasets. Entonces, estos tamaños iniciales diferentes logran un cambio en el campo de visión pero no en el tamaño real de la ventana, como se muestra en la Figura 3.2. El tamaño se indicará en las secciones correspondientes a cada dataset.

3.1.3.2. Duraciones de muestras

Otro parámetro fundamental es la cantidad de frames utilizados en cada muestra de los datasets. Este parámetro es el que marca la diferencia entre

utilizar o no (y en qué grado) información temporal como entrada de los modelos utilizados.

Las cantidades de frames utilizadas son 1, 3, 5 y 10 frames. Además, cada frame dista del otro en 0,5 segundos (2 FPS), es decir que las duraciones de las muestras son de 0,5, 1,5, 2,5 y 5 segundos. Se utilizan 0,5 segundos como distancia entre frames porque la consideramos una separación en la que no se pierde información (no es demasiado grande), ni se toman frames consecutivos muy similares entre sí (no es demasiado chica). Es decir que se busca un equilibrio entre pérdida y repetición de información, ya que la idea es que las muestras tengan temporalmente el mayor tamaño posible, pero a la vez deben ser soportadas por los métodos de aprendizaje automatizado utilizados en el trabajo, el cual es el motivo para limitar a 10 la cantidad de frames.

3.1.3.3. Dataset U: Rugby no etiquetado, por ventanas

Para entrenar a las DCGANs, se generan datasets que se componen de pequeñas ventanas espaciales y temporales de videos. Estas ventanas son recortes en la superficie de los videos (con los tres campos visuales ya mencionados) y limitados en la cantidad de frames (con las cuatro cantidades ya mencionadas), tal como se indica en 3.1.3.1 y 3.1.3.2.

Estas ventanas son tomadas al azar en espacio y tiempo de los videos que componen el primer grupo hasta llegar a 100000 muestras.

Cada muestra del dataset tendrá tamaño $(3 * Fr) \times 64 \times 64$, donde 3 es la cantidad de canales por frame (RGB), Fr es la cantidad de frames (1, 3, 5 o 10) y 64 es el ancho y alto del recorte. Este último valor se mantiene así para los tres campos visuales (mediante escalado), debido a que es el tamaño de la capa de entrada de la arquitectura que se utilizará en algunos de los experimentos.

De esta manera se creará el dataset U (*Unlabeled*), el cual es en realidad un conjunto de doce datasets, formados por la combinación de los tres tamaños de campos visuales y las cuatro duraciones.

3.1.3.4. Dataset L: Rugby etiquetado

Del grupo de videos etiquetados se forman tres subgrupos, tomando videos al azar y poniendo como restricción que dos videos de un mismo partido no puedan estar en grupos diferentes. Corresponden a los grupos entrenamiento,

validación y prueba (*test*), descriptos en 2.4. Las estadísticas de estos tres grupos son las siguientes:

Subdataset	**Nº de videos**	**Dur. promedio**	**Dur. total**
Entrenamiento	62	$2,9$ mins.	$180,6$ mins.
Validación	18	$2,8$ mins.	$50,5$ mins.
Prueba	18	$2,9$ mins.	$52,0$ mins.

Cuadro 3.3: Estadísticas de las subdivisiones del dataset de videos etiquetados.

Esta misma división en subgrupos se utilizará para los dos datasets descriptos a continuación, ya que para comparar de manera adecuada sus resultados, ambos deben utilizar la misma partición de datos de entrenamiento y de prueba.

3.1.3.4.1. Dataset L1: Rugby etiquetado, por ventanas

Para generar el primer dataset derivado de L, al cual llamaremos L1 (*Labeled*1), se toma la superficie de cada video y se realizan 9 recortes uniformemente distribuidos en una grilla de 3×3, como se ve en la Figura 3.3. La variable t toma los valores de tamaño de campo visual CE, ME y LE (explicados en la sección 3.1.3.1), generando así varios datasets. Cuando t toma los valores más altos, las ventanas generadas tienen áreas de la imagen en común (se intersecan). Esto no genera inconvenientes e incluso es algo deseable, ya que cubre mayor superficie del video. Antes de finalizar la construcción de los datasets, cada recorte es escalado para obtener una superficie de 64 px. $\times$ 64 px. De esta manera se define el área que abarcan las ventanas en cada video.

En cuanto a los frames incluidos, se toman ventanas con una distancia de 1 segundo entre el comienzo de una y la siguiente. La cantidad de frames abarcados varía entre los valores 1, 3, 5 y 10, y cada uno dista del otro en 0,5 segundos, es decir, a una tasa de 2 *FPS* (ver sección 3.1.3.2). Esto se puede observar en la Figura 3.4.

Cada muestra del dataset que se está construyendo contiene entonces 9 ventanas que ocupan diferente superficie del video (posiblemente con algo de superposición) pero que provienen de los mismos frames, es decir que están

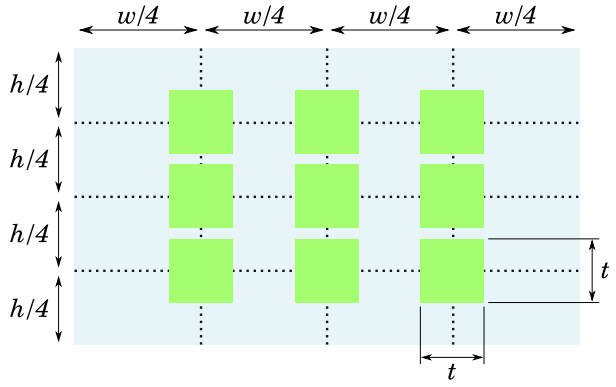


Figura 3.3: Forma de la grilla utilizada para recortar las ventanas del dataset L1. Los valores que toma t son $w/8$, $w/4$ o $w/2$, en función del campo visual buscado, donde $w \times h$ son las dimensiones del video. Luego estos recortes se escalan a 64 px. $\times$ 64 px., como se explica en 3.1.3.1.

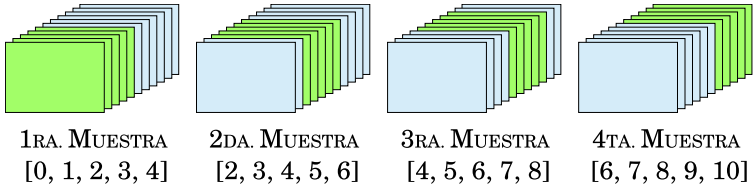


Figura 3.4: Eje temporal de las muestras tomadas de los videos. En este ejemplo se toman ventanas de 5 frames de duración.

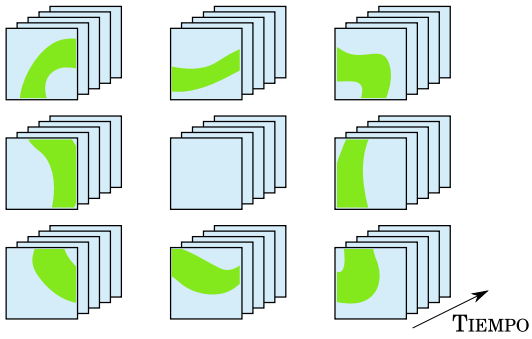


Figura 3.5: Estructura de una muestra completa del dataset L1. En este ejemplo, las muestras tienen una duración de 5 frames.

ubicados en el mismo segmento de la línea temporal del video. En la Figura 3.5 se observa la morfología de una muestra del dataset.

Finalmente, para cada video del dataset se toman todas las muestras que admita su duración, completando así el nuevo dataset.

Cada muestra del dataset tendrá tamaño $9 \times (3 * Fr) \times 64 \times 64$, donde 9 es la cantidad de ventanas de la muestra, 3 es la cantidad de canales por frame (RGB), Fr es la cantidad de frames (1, 3, 5 o 10) y 64 es el ancho y alto del recorte. Es importante ver que cada muestra está compuesta por nueve submuestras de tamaño $(3 * Fr) \times 64 \times 64$, el cual es idéntico al tamaño de cada muestra del dataset U.

Al igual que U, L1 es en realidad un conjunto de doce datasets, debido a las diferentes combinaciones entre tamaños de campos visuales y duraciones de ventanas.

Estas son las cantidades de muestras del dataset en función del número de frames (el tamaño de las ventanas no influye):

Nº. Frames	**Entrenamiento**	**Validación**	**Prueba**
1	10889	3048	3138
3	10827	3030	3120
5	10765	3012	3102
10	10587	2958	3048

Cuadro 3.4: Tamaños de los datasets en función de la cantidad de frames.

3.1.3.4.2. Dataset L2: Rugby etiquetado, región completa

L2 (*Labeled*2) es el segundo dataset derivado de L. A diferencia de L1, se compone de frames completos en lugar de recortes.

Debido a que las redes se entrenarán para tres tamaños de campos visuales diferentes (ver sección 3.1.3.1), las imágenes del dataset deben estar disponibles en 512 px. de ancho para el campo visual CE, 256 px. para ME y 128 px. para LE.

Para determinar la cantidad de frames de cada muestra y el inicio de cada una, se utilizaron las mismas reglas que en el dataset L1: muestras con duraciones de 1, 3, 5 y 10 frames, donde cada una dista de la otra en 0,5 segundos (2 FPS). Esto se encuentra detallado en 3.1.3.2. Además, cada muestra inicia 2 frames (1 segundo) después del inicio de la anterior. Esto puede apreciarse en la Figura 3.4.

Cada muestra del dataset tendrá tamaño $(3 * Fr) \times ImgSize \times ImgSize$, donde 3 es la cantidad de canales por frame (RGB), Fr es la cantidad de frames (1, 3, 5 o 10) e $ImgSize$ toma los valores 512, 256 o 128, para los campos de visión CE, ME y LE respectivamente.

La cantidad de ejemplos del dataset para cada cantidad de frames coincide exactamente con las cantidades del dataset L1, debido a que se utilizaron las mismas reglas para elegir los frames al componer las muestras (aunque espacialmente los datasets son diferentes, temporalmente son idénticos).

3.1.3.4.2.1. Relación de aspecto

Como la entrada del modelo de red convolucional que hará uso de este dataset es cuadrada, pero los videos con los que contamos no lo son (en su mayoría tienen relación 4:3, 16:9 o similar), los mismos deben adaptarse a esta relación. Las alternativas que se probarán y que, por lo tanto, deben estar incluidas en este dataset son dos:

SQ Se estiran/aplastan las imágenes, produciendo una distorsión que *podría* afectar negativamente los resultados del modelo, pero con la ventaja de que se aprovecha toda la superficie que la red está procesando. A esta alternativa la llamaremos SQ (*square*).

16:9 Se coloca la imagen dentro del cuadrado, centrada y redimensionada de manera que ésta ocupe la mayor superficie posible, pero respetando la relación de aspecto original de la misma. Donde no hay imagen, se

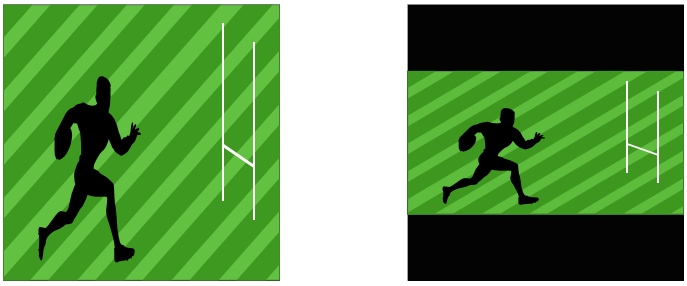


Figura 3.6: Representaciones de un mismo frame con relación de aspecto SQ y 16:9. A la izquierda (SQ), la imagen ocupa la totalidad de la superficie disponible, pero se encuentra levemente distorsionada, ya que pierde la relación original de alto-ancho. A la derecha (16:9), la imagen preserva su relación de aspecto, pero no utiliza la totalidad de la superficie, por lo que ese espacio es ocupado por dos franjas negras.

pintan los píxeles de negro. La razón por la que se hace esto es la similitud que presenta con el llamado *zero-padding*, utilizado comúnmente en las redes convolucionales y explicado en la sección 2.7.3.1. Nos referiremos a esta opción como 16:9 (aunque no todos los videos tengan exactamente dicha relación).

En la Figura 3.6 puede verse un esquema que ilustra la diferencia entre ambos abordajes.

Debido a esta nueva variable, L2 es entonces un grupo de veinticuatro datasets, en lugar de doce como los anteriores, ya que las doce combinaciones de tamaños de campos visuales y duraciones (utilizadas también en los datasets anteriores) se duplican al agregar la relación de aspecto entre las variantes que debe contener el dataset.

3.2. Experimentos

En este capítulo detallaremos el proceso experimental de todo el trabajo. Sus secciones comprenden los siguientes procedimientos:

- La construcción de la red DCGAN y su posterior entrenamiento utilizando videos no etiquetados, con el objetivo de ver qué tan bien la

red aprende de este tipo de datos y de qué manera las muestras son resumidas por el discriminador.

- La utilización del discriminador, junto con modelos de SVM para la clasificación de fragmentos de videos etiquetados. Medición de los resultados de algunos modelos, combinados con diferentes formas de generar muestras para los datasets a partir de los videos.
- El entrenamiento de modelos que utilizan redes convolucionales para clasificar fragmentos de videos etiquetados, persiguiendo el mismo objetivo que al utilizar SVMs. También incluye la medición de resultados obtenidos.
- La realización de un entrenamiento similar al anterior, pero inicializando los pesos de las neuronas de las capas convolucionales con los pesos aprendidos por la red DCGAN (en el discriminador), para ver si se obtienen mejores resultados al aprovechar este aprendizaje previo con datos no etiquetados.
- La prueba de diferentes métodos de generalización para clasificar videos completos utilizando resultados de clasificación de ventanas de entre 0,5 y 5 segundos.

3.2.1. Entrenamiento de DCGAN

Uno de los objetivos de este trabajo consiste en utilizar GANs para generar, de manera no supervisada, una representación común (en forma de vector numérico) para pequeñas ventanas temporales y espaciales de videos. Para esto, se entrenarán algunos modelos de este tipo de red y de los mismos se tomarán los discriminadores. En el proceso obtendremos también generadores, los cuales utilizaremos para decidir cuándo detener el proceso de entrenamiento.

3.2.1.1. Construcción del modelo

La arquitectura de las redes empleadas en este capítulo está basada en la utilizada por A. Radford et al. [2]. En dicho trabajo usaron este tipo de redes para generar imágenes de dormitorios.

Como se explica en 2.7.4, las redes GAN están compuestas por un generador y un discriminador. A continuación se describe la estructura que se

le da a cada uno en este trabajo. Llamaremos *Ch* a la cantidad de canales de las imágenes utilizadas como entrada de la red. Los posibles valores que puede tomar este parámetro son 3, 9, 15 y 30, debido a que la cantidad de frames puede variar entre 1, 3, 5 y 10; y a que cada frame se compone de tres canales de colores (rojo, verde y azul).

3.2.1.1.1. Generador

Como se explicó en la sección 2.7.4, el generador es la porción de la red encargada de producir imágenes utilizando como entrada vectores de ruido aleatorio con distribución uniforme. Por ello su capa de entrada consta de 100 neuronas, cuya disposición no tiene correspondencia espacial con las imágenes producidas. Su salida está compuesta por un bloque con el ancho y alto de las imágenes producidas (64×64) y una profundidad igual a la cantidad total de canales de las muestras (*Ch*). La arquitectura completa del generador utilizado es la siguiente:

Capa	**Nº filtros**	**Tamaño filtros**	***Stride***	**Tamaño salida**
Entrada				100
Compl. Conect. ($1024 * 4 * 4$ unidades)				$1024 \times 4 \times 4$
Normalización				
No-linealidad (ReLU)				
Conv. T.	512	5×5	2×2	$512 \times 8 \times 8$
Normalización				
No-linealidad (ReLU)				
Conv. T.	256	5×5	2×2	$256 \times 16 \times 16$
Normalización				
No-linealidad (ReLU)				
Conv. T.	128	5×5	2×2	$128 \times 32 \times 32$
Normalización				
No-linealidad (ReLU)				
Conv. T.	Ch	5×5	2×2	$Ch \times 64 \times 64$
No-linealidad (TanH)				

Cuadro 3.5: Arquitectura del generador.

3.2.1.1.2. Discriminador

El discriminador toma como entrada las imágenes, y en cada capa posterior va reduciendo sus dimensiones espaciales, pero a su vez aumentando la cantidad de filtros, hasta convertir la entrada en un vector con 1024 componentes. Este vector finalmente es procesado por una capa densa, con una sola neurona de salida, que se activa o no dependiendo de que la imagen sea considerada una muestra real o una producida por el generador.

Capa	**Nº filtros**	**Tamaño filtros**	***Stride***	**Tamaño salida**
Entrada				Ch × 64 × 64
Conv.	128	5 × 5	2 × 2	128 × 32 × 32
Normalización				
No-linealidad (LReLU)				
Conv.	256	5 × 5	2 × 2	256 × 16 × 16
Normalización				
No-linealidad (LReLU)				
Conv.	512	5 × 5	2 × 2	512 × 8 × 8
Normalización				
No-linealidad (LReLU)				
Conv.	1024	5 × 5	2 × 2	1024 × 4 × 4
Normalización				
No-linealidad (LReLU)				
Pooling (máximo)				1024
Compl. Conect. (1 unidad)				1
No-linealidad (Sigmoide)				

Cuadro 3.6: Arquitectura del discriminador.

3.2.1.1.3. Implementación

Los experimentos se realizaron utilizando Python como lenguaje de programación, junto con la librería Lasagne para construir y entrenar las redes sobre Theano. El entrenamiento se realizó sobre PCs que cuentan con GPUs modelo GeForce GTX 1070.

3.2.1.2. Entrenamiento

La red es inicializada con pesos aleatorios, con distribución normal y desvío estándar $0,02$. El entrenamiento se realiza utilizando el algoritmo Adam (ver 2.7.2.5.4), con los siguientes hiperparámetros: *learning rate* = $0,0002$, $\beta_1 = 0,5$, $\beta_2 = 0,999$ (valores de *weight decay*) y tamaño de lote = 128. Para calcular la pérdida (*loss*) se utiliza entropía cruzada binaria (*binary cross entropy*). Las muestras se normalizan en el rango (-1, 1).

Durante el entrenamiento, cada 1000 iteraciones se guardan muestras producidas por el generador (artificiales) para tener una noción de la calidad de las imágenes que se están produciendo. Además, se almacena la pérdida del discriminador y del generador en esa iteración.

3.2.1.3. Generación de nuevos datos de la red

En la Figura 3.7 se muestran frames producidos por los generadores de algunas redes, utilizando vectores aleatorios como entradas de los mismos. En el Apéndice A se muestran más ejemplos, para visualizar por un lado la calidad de los frames individualmente (sección A.1) y por el otro la calidad del movimiento producido (sección A.2).

3.2.1.3.1. Inconvenientes durante el entrenamiento

Al analizar las imágenes producidas, nos encontramos con que en ocasiones el entrenamiento de la red se desestabiliza, y la misma comienza a producir imágenes similares para diferentes vectores de ruido aleatorio utilizados como entrada del generador. Un ejemplo de esto puede observarse en la Figura 3.8.

Otro problema que se presenta ocasionalmente durante el entrenamiento provoca que las imágenes producidas pierdan contraste, volviéndose cada vez más grises (alejándose de esta manera de la distribución de las muestras reales). Este inconveniente no ocurre de forma brusca como en el caso anterior, sino que va empeorando progresivamente a lo largo de las iteraciones. En la sección A.3 del Apéndice A se puede observar parte de un entrenamiento en el que se presenta este problema.

Para evitar estos inconvenientes el entrenamiento se detiene antes de que los mismos se presenten. Con este fin se almacenan los modelos anteriores y cuando alguno de estos dos problemas se hace visible se utiliza el modelo guardado antes de que esto ocurra.



Figura 3.7: Imágenes artificiales producidas por los generadores de las redes de 3 frames (columna izquierda) y 5 frames (columna derecha), con campos visuales CE (arriba), ME (centro) y LE (abajo). Se visualiza únicamente el primer frame de cada muestra (ver Apéndice A para más ejemplos).



Figura 3.8: Imágenes producidas por una red cuyo proceso de entrenamiento se desestabilizó. Muchas de estas imágenes son casi iguales entre sí, a pesar de haber sido generadas a partir de diferentes vectores aleatorios.

3.2.1.4. Extracción del discriminador

Nuestro objetivo al entrenar la red GAN es extraer un modelo que tome como entrada una imagen y le aplique una transformación que descarte los detalles irrelevantes de la misma, reduciendo la cantidad de información complementaria que no nos ayude a diferenciar una clase de otra. Esto nos deja en su salida un vector con la información útil codificada.

Podemos ver que el discriminador (sección 3.2.1.1.2) toma una imagen como entrada (tal cual se busca) y en su salida posee una capa densa con una sola neurona. Esta neurona se activará cuando la red considere a la imagen de entrada real y no lo hará cuando la considere sintetizada por el generador.

Si a nuestro discriminador (cuya capa anterior a la última posee 1024 neuronas) le quitamos la última capa, quedándonos con la parte que se extiende desde su capa de entrada hasta la penúltima capa, obtendremos una red que, en principio, puede ser lo que estamos buscando. A esta red, cuya salida será un vector de 1024 componentes en lugar de un escalar, la llamaremos D' (ver Figura 3.9).

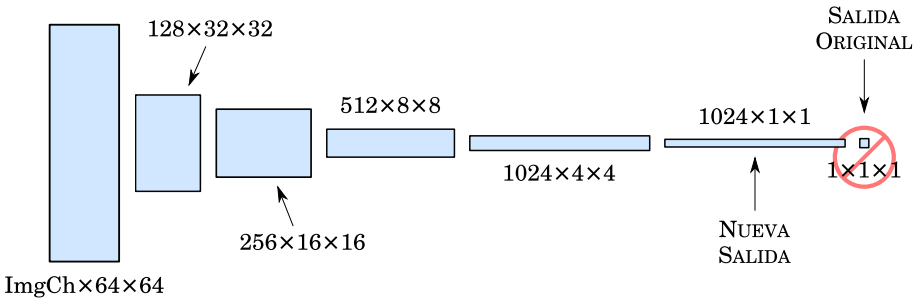


Figura 3.9: Arquitectura de la red D'. Su salida es un vector de 1024 componentes en lugar de un escalar.

3.2.1.5. Previsualización de resultados mediante PCA e histogramas

A continuación, intentaremos ver cómo el modelo mencionado anteriormente está transformando las imágenes de entrada en vectores, qué es lo que está considerando relevante y qué no. Para ello, se aplican algunos métodos de agrupamiento (*clustering*) sobre estos vectores y se visualizan sus resultados, de la siguiente manera:

1. Se aplica el modelo a todas las ventanas del dataset U (sección 3.1.3.3), y las salidas son recolectadas en un nuevo dataset U', el cual contendrá las representaciones de dichas ventanas en forma de vector numérico. Si la forma del dataset U era (N × Ch × 64 × 64), la de U' será (N × 1024).

2. Se realiza PCA sobre el dataset U'.

3. *a*) Se aplica k-means con $k = 2$ sobre ese resultado.

   b) Se generan los histogramas de los 10 primeros componentes de PCA (un histograma por cada componente).

4. Para cada partición generada en *a* y *b* se toman aleatoriamente imágenes de diferentes clusters, para analizar la separación que realizó k-means sobre los datos (*a*) y tener una visualización de qué están representando los diferentes componentes, es decir, qué significado tiene cada uno (*b*). Recordemos que hasta este momento no se realizó ningún

entrenamiento supervisado, por lo que no es de esperar que las clases encontradas tengan una correspondencia con las clases buscadas.

Finalmente, para tener un punto de comparación, se realiza el mismo procedimiento directamente sobre las imágenes del dataset, tomando el valor de cada píxel para cada canal como un atributo de la muestra. En este caso el procedimiento es el mismo, con la excepción de que se omite el primer paso. Se realiza únicamente con el dataset de campo visual CE y 1 frame de duración, ya que la mayoría de los resultados que se mostrarán para el caso anterior (vectores representación) corresponden a este dataset[1].

3.2.1.5.1. Resultados del *clustering* sobre vectores representación

La siguiente recopilación de imágenes con sus correspondientes descripciones y comentarios no pretende ser exhaustiva, sino sólo una muestra de la capacidad de los modelos entrenados de manera no supervisada para resumir secuencias de imágenes como vectores numéricos.

3.2.1.5.1.1. Clusters encontrados con *k*-means

Para todas las combinaciones de tamaño de campo visual y cantidad de frames, encontramos que los dos clusters que encuentra k-means podrían describirse como "imágenes claras" e "imágenes oscuras". Esto es lo esperado, ya que concuerda con el primer componente principal, como veremos a continuación. En la Figura 3.10 pueden verse algunos ejemplos.

3.2.1.5.1.2. Clusters definidos por histogramas

En todos los casos vistos (para campos visuales CE, ME y LE, y distintas cantidades de frames) el primer componente siempre estuvo ligado a la iluminación de las imágenes. Un ejemplo de esto puede verse en la Figura 3.11, siendo todos los casos muy similares a éste.

Casi siempre, el segundo componente (o al menos uno de los primeros, dependiendo del modelo) presenta un comportamiento en el que las imágenes en uno de los extremos son en general imágenes de césped, casi sin ningún otro elemento, mientras que en el otro extremo son imágenes muy complejas,

[1]Esto se debe a que al momento de redactar el trabajo se comenzó por el mismo y luego los clusters hallados en los restantes datasets son parecidos.



CE (Cerca), 1 Frame



ME (Medio), 10 Frames



LE (Lejos), 10 Frames

Figura 3.10: Clusters encontrados por k-means, para CE, ME y LE.



Figura 3.11: De izquierda a derecha, imágenes tomadas al azar de cada uno de los diez bins del histograma definido por el componente 0 de PCA, con campo visual CE y muestras de 1 frame de duración. En la Figura B.1 del Apéndice B se encuentra ampliado.

compuestas por muchos objetos u objetos con muchos detalles. En la Figura 3.12 se muestran imágenes de cinco clusters generados a partir del histograma de un componente con las características mencionadas.

A partir del tercer componente, los resultados varían entre los diferentes tamaños de ventanas y cantidad de frames. Esto puede apreciarse en la Figura 3.13, donde se presentan imágenes tomadas al azar de algunos clusters encontrados utilizando estos componentes.

3.2.1.5.2. Resultados del *clustering* sobre píxeles

Los clusters hallados con k-means y los hallados con el histograma del primer componente principal son parecidos a los encontrados en el caso anterior (vectores representación), por ello no se muestran aquí. Recordemos que k-means con $k = 2$ encuentra dos clusters tales que uno contiene imágenes claras y el otro contiene imágenes oscuras; y en el histograma los bins van desde las imágenes muy oscuras hasta las muy claras.

Por otro lado, al analizar los clusters determinados por el histograma del segundo componente y los siguientes, se encuentran diferencias respecto al método anterior. Estos resultados se muestran desde la Figura 3.14 hasta la Figura 3.18.

3.2.2. Predicción con DCGAN + SVMs

En esta sección utilizaremos por primera vez uno de los datasets con muestras etiquetadas. En estas pruebas se utilizará el modelo D' de la sección 3.2.1.4, en conjunto con clasificadores SVM.



Figura 3.12: De izquierda a derecha, imágenes tomadas al azar de cinco bins centrales del histograma del componente 1, generado con campo visual CE, 1 frame.



Figura 3.13: Algunos grupos hallados al aplicar PCA e histogramas a los vectores representación. Se pueden ver ejemplos más completos y detallados desde la Figura B.2 hasta la Figura B.20 del Apéndice B.



Figura 3.14: Muestras de los bins del histograma de un componente generado al aplicar PCA sobre los píxeles. En los primeros bins se encuentran zonas oscuras en la parte inferior de las imágenes y zonas claras en la parte superior, mientras que en los últimos ocurre lo opuesto.



Figura 3.15: Muestras de los bins del histograma de un componente generado al aplicar PCA sobre los píxeles. En los primeros bins las imágenes se componen de colores cercanos al azul y en los últimos de colores cercanos al verde, en los bins centrales se encuentran el resto de las imágenes.



Figura 3.16: Muestras de los bins del histograma de un componente generado al aplicar PCA sobre los píxeles. En los primeros bins se encuentran zonas oscuras en la parte izquierda de las imágenes y zonas claras en la parte derecha, mientras que en los últimos ocurre lo contrario.



Figura 3.17: Muestras de los bins del histograma de un componente generado al aplicar PCA sobre los píxeles. En los primeros bins las imágenes se componen de colores cercanos al rojo y en los últimos de colores cercanos al azul, en los bins centrales se encuentran el resto de las imágenes.



Figura 3.18: Muestras de los bins del histograma de un componente generado al aplicar PCA sobre los píxeles. Las imágenes de los primeros bins poseen colores claros —casi blancos— en el centro, mientras que las imágenes de los últimos bins poseen colores claros a los costados y oscuros en el centro.

3.2.2.1. Ejecución de red discriminador sobre datos

De manera análoga a como se hizo en la sección 3.2.1.5.1 para crear U' a partir de U, se generará L1' a partir de L1 (es decir, aplicando D' a cada ventana del dataset). En este caso, como cada muestra de L1 no es una sola ventana, sino que es un conjunto de 9 ventanas, en lugar de obtener un vector por cada muestra, obtendremos 9. De esta forma, si el tamaño del dataset L1 es (N × 9 × Ch × 64 × 64), el del nuevo dataset será (N × 9 × 1024).

3.2.2.2. Support vector machines

A continuación, serán entrenados modelos de SVM para clasificar muestras del dataset L1'. Se probarán modelos variando el kernel, algunos parámetros de SVM y preprocesando los datos de diferentes maneras.

3.2.2.2.1. Preprocesamiento de los datos

Debido a que cada muestra del dataset L1' está compuesta por 9 vectores, debemos escoger la manera de transformarlos en una única entrada para los modelos SVM que utilizaremos, para lo cual se probarán diferentes alternativas, algunas simples y otras un poco más complejas.

Sean $v = (v_0, v_1, ..., v_m)$ y $w = (w_0, w_1, ..., w_n)$ dos vectores finitos, se nota la operación $v +\!\!+ w$ como la concatenación de ambos vectores, es decir que

$$v +\!\!+ w = (v_0, v_1, ..., v_m, w_0, w_1, ..., w_n).$$

Dicho esto, y asumiendo que los 9 vectores en cuestión son llamados $v_0, v_1, ..., v_8$, donde $v_i = (v_{i,0}, v_{i,1}, ..., v_{i,1023})$, con $i \in \{0, 1, ..., 8\}$, las alternativas que se pondrán a prueba son las siguientes (todas incluyen normalización):

- **Concat**: es simplemente la concatenación de los 9 vectores de 1024 componentes, para formar un solo vector de 9216 componentes. Su salida entonces será
$$v_0 +\!\!+ v_1 +\!\!+ ... +\!\!+ v_8.$$
- **Max**: consiste en asignar a cada componente j del vector resultado el máximo valor entre los j-ésimos componentes de todos los vectores, dando como resultado
$$(m_0, m_1, ..., m_{1023}), \text{ donde } m_j = \max_{i=0}^{8} v_{i,j}.$$

- **Min**: es similar a **Max**, pero utilizando el mínimo en lugar del máximo, por lo que su salida será

$$(m'_0, m'_1, ..., m'_{1023}), \text{ donde } m'_j = \min_{i=0}^{8} v_{i,j}.$$

- **Max+Min**: consiste en concatenar el resultado de los dos procedimientos anteriores, es decir,

$$\mathbf{Max}(v_0, v_1, ..., v_8) +\!+ \mathbf{Min}(v_0, v_1, ..., v_8).$$

- **PCA$_i$(Concat)**, con $i \in \{3, 16, 128, 1024\}$: la cual aplica el algoritmo de reducción de dimensionalidad PCA, manteniendo i componentes, al resultado del método **Concat**.

- **PCA$_i$(Max+Min)**, con $i \in \{3, 16, 128, 1024\}$: es similar a **PCA$_i$(Concat)**, pero aplica PCA a la salida de **Max+Min**, en lugar de a la salida de **Concat**.

Los métodos de preprocesamiento descriptos tendrán los siguientes tamaños de salida por muestra:

Método	**Tamaño salida**	**Observaciones**
Concat	9216	$9 * 1024$
Max	1024	—
Min	1024	—
Max+Min	2048	$1024 + 1024$
PCA$_\text{i}$(Concat)	i	3/16/128/1024
PCA$_\text{i}$(Max+Min)	i	3/16/128/1024

Cuadro 3.7: Tamaños de salida de los métodos de preprocesamiento.

3.2.2.2.2. Construcción de los modelos

Trabajaremos con dos modelos diferentes:

SVM$_{\text{lineal}}$ Support vector machines con kernel lineal y soporte multiclase con el esquema "uno vs. el resto".

$\mathbf{SVM_{RBF}}$ Support vector machines con kernel *radial basis function* y soporte multiclase con el esquema "uno vs. uno".

La elección de dichos esquemas para el soporte multiclase de cada modelo se debe a que son respectivamente los únicos soportados por las implementaciones utilizadas (ambas de scikit-learn).

Se hará una búsqueda de hiperparámetros por medio de grilla con los siguientes valores de C y *gamma* (γ):

Hiperp.	**Valores**
C	10^{-3}, 10^{-2}, ..., 10^{6}, 10^{7}
γ	10^{-5}, 10^{-4}, ..., 10^{2}, 10^{3}, 'auto'

Cuadro 3.8: Valores que toman los hiperparámetros durante la búsqueda, donde '*auto*' $= 1/n$ y n es la cantidad de atributos del dataset. El hiperparámetro γ sólo es utilizado por $\mathrm{SVM_{RBF}}$.

3.2.2.2.2.1. Implementación

El trabajo con redes de esta sección se hizo bajo las mismas condiciones que en 3.2.1.1.3 (Python + Lasagne). El trabajo con SVMs y con PCA y la normalización de los datos se hicieron utilizando la librería scikit-learn, también en Python. De dicha librería, se utilizó la clase StandardScaler para centrar y escalar los datos, la clase PCA para el aplicar el algoritmo del mismo nombre, la clase LinearSVC para el SVM clasificador con kernel lineal y la clase SVC para el SVM con kernel RBF. Estos algoritmos corrieron utilizando una PC con CPU Intel® Core™ i7-4790K @ 4.00GHz con 8GB RAM.

3.2.2.2.3. Entrenamiento

Debido a que vamos a probar doce formas diferentes de preprocesar los datos, junto con dos modelos donde cada uno será parametrizado con once valores de C, y uno de ellos además con diez valores de gamma, no probaremos todas las posibles combinaciones ya que son 1452. Cada una de ellas, además, sería entrenada para los 12 datasets que componen L1, dando un total de 17424 entrenamientos. Suponiendo que cada entrenamiento toma aproximadamente entre 15 y 20 minutos (en realidad suelen ser más largos), eso nos da un tiempo total de entrenamiento de al menos 180 días.

Para no requerir semejante cantidad de tiempo, primero se realizaron entrenamientos con todos los preprocesamientos propuestos y los dos kernels con valores por defecto para los hiperparámetros ($C = 1$ y $\gamma =$ '*auto*'), para seleccionar el mejor preprocesamiento y kernel. A continuación se seleccionaron los valores óptimos para los hiperparámetros por medio de una búsqueda en grilla.

La selección de hiperparámetros se hizo en ese orden para mantener casi hasta el final las diferentes alternativas de cantidad de frames y tamaño de campo visual, hasta el punto previo al ajuste más fino (elección de *gamma*).

3.2.2.2.3.1. Tiempos de entrenamiento

En total se realizaron tres etapas de entrenamiento:

- **1ra** Elección de preprocesamiento y kernel.
- **2da** Elección de C, tamaño de campo visual y cantidad de frames.
- **3ra** Elección de *gamma*.

Los tiempos de entrenamiento de cada etapa, junto con la cantidad de modelos entrenados en cada una, son los siguientes:

Etapa	Modelos	Tiempo total	Tiempo promedio
1ra	96	$35,8$ hs.	$0,4$ hs.
2da	44	$116,7$ hs.	$2,7$ hs.
3ra	10	$9,4$ hs.	$0,9$ hs.
Total	150	$161,9$ hs.	$1,1$ hs.

Cuadro 3.9: Tiempos de entrenamiento por etapa.

El tiempo total de entrenamiento fue de casi 6 días y 18 horas.

3.2.2.2.4. Resultados

Primero mostraremos una comparación entre los resultados obtenidos al utilizar los diferentes tamaños de campo visual y cantidades de frames. Luego mostraremos los hiperparámetros escogidos en base a la búsqueda realizada.

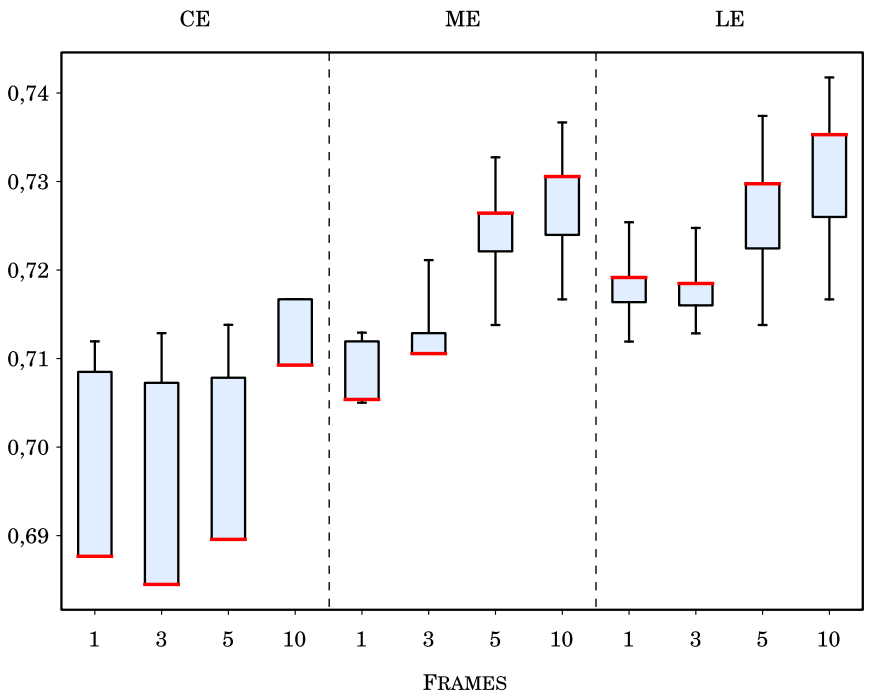


Figura 3.19: *Accuracies* obtenidas por los modelos de SVM al utilizar diferentes tamaños de campo visual y cantidades de frames. Las cajas se extienden desde el cuartil inferior hasta el cuartil superior, con una línea roja sobre la mediana. Los bigotes muestran el rango de los datos. Notoriamente, las medianas se ubican en los extremos de las cajas.

3.2.2.2.4.1. Comparación por tamaño de campo visual y cantidad de frames

En la Figura 3.19 se pueden ver los resultados obtenidos al variar el tamaño de campo visual y la cantidad de frames por muestra. Las mejores *accuracies* obtenidas por cada combinación de estos dos hiperparámetros se muestran en el Cuadro 3.10, ordenadas de modo tal que se pueda apreciar fácilmente la dirección en que mejoran las mismas. Utilizando estos resultados se fijaron los valores para ambos hiperparámetros.

3.2.2.2.4.2. Modelo seleccionado

En base a los resultados, el modelo de SVM elegido es el siguiente:

	CE	ME	LE
1 Frame	0,712	0,713	0,725
3 Frames	0,713	0,721	0,725
5 Frames	0,714	0,733	0,737
10 Frames	0,717	0,737	0,742

Cuadro 3.10: Mejores *accuracies* obtenidas por cada combinación de campo visual y cantidad de frames. Las mismas aumentan al incrementar la cantidad de frames o alejar el campo visual.

- **Preprocesamiento:** Concat (concatenación de los 9 vectores, dejando 9×1024 componentes)
- **Kernel:** RBF
- **C:** 1
- **Gamma (γ):** 10^{-4}
- **Tamaño de campo visual de ventana:** LE
- **Cantidad de frames:** 10

Este modelo obtuvo las siguientes *accuracies* para los datos utilizados:

- **Validación:** $0,7438$
- **Prueba:** $0,7513$

3.2.3. Predicción con redes convolucionales

En esta sección intentaremos utilizar redes convolucionales para realizar la misma tarea que en la sección anterior. Para ello, haremos uso del dataset L2 (descrito en sección 3.1.3.4.2), el cual contiene frames completos en lugar de recortes espaciales de los videos. Utilizaremos las convoluciones propias de este tipo de redes para recorrer toda la superficie de los fragmentos de video. Al no utilizarse un modelo clasificador como complemento, la salida de la red indicará en este caso la clase predicha.

3.2.3.1. Construcción del modelo

Como en la sección 3.2.1.1, llamaremos *Ch* a la cantidad de canales de las imágenes de entrada. Este parámetro tomará los valores 3, 9, 15 y 30, debido a que la cantidad de frames será 1, 3, 5 y 10; y cada frame se compone de tres canales.

Size, por otro lado, corresponde al tamaño en píxeles de las imágenes (alto y ancho, ya que son cuadradas). Los valores que tomará son 512, 256 y 128, correspondientes a los tamaños de campo visual de ventana CE, ME y LE.

La estructura de la red es la siguiente:

Capa	**Nº filtros**	**Tamaño filtros**	***Stride***	**Tamaño salida**
Entrada				Ch × Size × Size
Conv.	128	5 × 5	2 × 2	128 × Size/2 × Size/2
Normalización				
No-linealidad (LReLU)				
Conv.	256	5 × 5	2 × 2	256 × Size/4 × Size/4
Normalización				
No-linealidad (LReLU)				
Conv.	512	5 × 5	2 × 2	512 × Size/8 × Size/8
Normalización				
No-linealidad (LReLU)				
Conv.	1024	5 × 5	2 × 2	1024 × Size/16 × Size/16
Normalización				
No-linealidad (LReLU)				
Pooling (máximo)				1024
Compl. Conect. (3 unidades)				3
No-linealidad (Softmax)				

Cuadro 3.11: Arquitectura de la red convolucional.

Los tamaños de salida efectivos de las capas, dependiendo del tamaño de campo visual de las imágenes de entrada utilizada, serán:

Capa	CE	ME	LE
Entrada	Ch × 512 × 512	Ch × 256 × 256	Ch × 128 × 128
Conv.	128 × 256 × 256	128 × 128 × 128	128 × 64 × 64
Conv.	256 × 128 × 128	256 × 64 × 64	256 × 32 × 32
Conv.	512 × 64 × 64	512 × 32 × 32	512 × 16 × 16
Conv.	1024 × 32 × 32	1024 × 16 × 16	1024 × 8 × 8
Pooling	1024	1024	1024
Compl.	3	3	3

Cuadro 3.12: Tamaños de salida de cada capa, en función del campo visual.

3.2.3.1.1. Convoluciones de la red

Cuando se entrenó el discriminador (con DCGAN), las convoluciones realizadas por la red barrían una superficie de 64 × 64 px. en la primera capa, mientras que en las siguientes 32 × 32 px., 16 × 16 px. y finalmente 8 × 8 px. en la última capa convolucional. Esto ocurre porque las imágenes de entrada tenían un tamaño de 64 px. de alto y de ancho.

En cambio, durante el entrenamiento y clasificación de estas redes, las entradas varían entre 128 px., 256 px. y 512 px. de alto y ancho. En la Figura 3.20 puede apreciarse la diferencia en la cantidad de pasos requeridos por la operación de convolución para cada tamaño de campo visual de ventana.

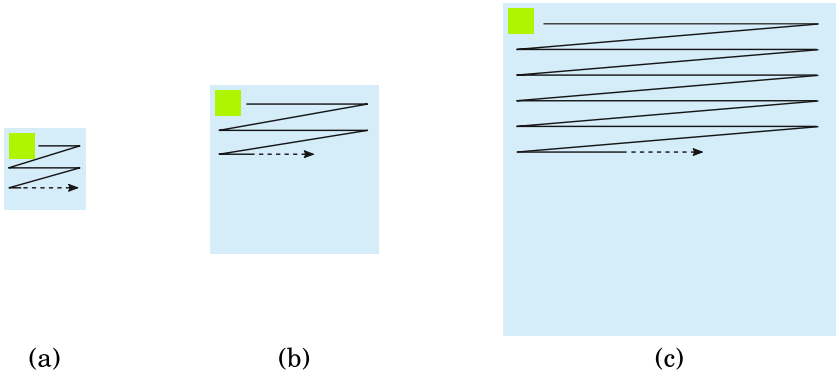


Figura 3.20: Convoluciones para los diferentes tamaños de campo visual de ventana: (a) ventanas con campo visual LE, (b) campo ME y (c) campo CE.

3.2.3.1.2. Implementación

Al igual que en los experimentos anteriores, las redes se construyeron y entrenaron con Lasagne, y se utilizaron PCs de similares características a las mencionadas en la sección 3.2.1.1.3.

3.2.3.2. Entrenamiento

El entrenamiento comienza con pesos iniciales aleatorios, con distribución normal y desvío estándar $0,02$. La actualización de los pesos se realiza mediante el algoritmo Adam (ver 2.7.2.5.4), con hiperparámetros *learning rate* = $0,0002$, $\beta_1 = 0,5$ y $\beta_2 = 0,999$ (valores de *weight decay*). Para calcular la pérdida (*loss*) se utiliza entropía cruzada para categorías (*categorical cross entropy*). Las muestras se normalizan en el rango (-1, 1).

Cada 20 iteraciones se mide la pérdida y los errores de entrenamiento, validación y prueba (éstos últimos no son utilizados hasta el final del trabajo).

3.2.3.3. Resultados

Los mejores modelos se lograron con campo visual CE y 5 frames para 16:9, y con campo visual CE y 10 frames para SQ. De estos dos modelos, el que mejor *accuracy* tuvo fue el segundo, utilizando la relación de aspecto SQ.

Sus resultados finales, para los conjuntos de datos de validación y prueba, son los siguientes:

- **Validación:** $0,7691$
- **Prueba:** $0,7644$

A continuación se comparan las *accuracies* obtenidas por los modelos que trabajan con los diferentes tamaños de campo visual de ventana, cantidades de frames y relaciones de aspecto. El objetivo es ver si mejoran las predicciones al aumentar la cantidad de frames, al reducir/acercar el campo visual de las ventanas y al utilizar SQ en lugar de 16:9. Para ello se calculan las medias aritméticas de las mejoras al utilizar los diferentes valores.

Hiperparámetro	Mejora *accuracy* (%)
LE	$0,00 \pm 0,00$
ME	$1,57 \pm 0,30$
CE	$2,76 \pm 0,27$
1 frame	$0,00 \pm 0,00$
3 frames	$0,55 \pm 0,55$
5 frames	$1,05 \pm 0,58$
10 frames	$1,89 \pm 0,37$
16:9	$0,00 \pm 0,00$
SQ	$1,96 \pm 0,27$

Cuadro 3.13: Mejoras en la *accuracy* al modificar el tamaño de campo visual, la cantidad de frames o la relación de aspecto. Cabe destacar que estos resultados corresponden a porcentajes, no a puntos porcentuales.

3.2.4. Predicción con redes convolucionales preentrenadas con DCGAN

Les redes convolucionales utilizadas en la sección anterior comparten (intencionalmente) algunas características con los discriminadores DCGAN de la sección 3.2.1.1.2, como la cantidad de filtros, su tamaño y el *stride*. A pesar de esto, los resultados del entrenamiento de las DCGANs no fueron empleados en estas redes.

En esta sección intentaremos utilizar lo aprendido por las redes DCGAN (en particular el discriminador) para mejorar el desempeño de las redes convolucionales. Usaremos la misma arquitectura, con los mismos datos y entrenando de la misma manera, pero inicializaremos los pesos de las redes convolucionales utilizando los pesos de los discriminadores de las redes DCGAN.

3.2.4.1. Construcción del modelo

La estructura de la red es exactamente la misma que en la sección 3.2.3.1, sólo difieren en la forma en que serán entrenadas.

Es importante notar que está compuesta por algunas capas convolucionales con filtros similares a los utilizados en el discriminador de DCGAN (mismos tamaños y *strides*). Aunque comparten las dimensiones y *strides* de

sus filtros, estas capas no tienen el mismo tamaño de salida, ya que están aplicados sobre entradas de diferente tamaño. Esto es posible porque los filtros utilizados por este tipo de capas y sus pesos son independientes del tamaño de imagen de entrada, haciendo que los mismos puedan utilizarse en otra red que trabaje con imágenes de diferentes dimensiones.

Las redes también difieren en su última capa. En ambos casos es una capa densa, pero en el caso del discriminador la salida consiste en sólo una neurona, donde la no-linealidad está dada por una función sigmoide, mientras que en la red de esta sección, la última capa posee tres neuronas, con función de activación softmax. En la Figura 3.21 puede verse un esquema que ilustra ambas redes.

3.2.4.1.1. Implementación

Todo lo referido a la implementación es igual a la sección anterior. Los pesos de las redes preentrenadas se almacenan y se leen del disco utilizando el formato npz, con los métodos numpy.savez y numpy.load.

3.2.4.2. Entrenamiento

El entrenamiento se realiza exactamente de la misma manera que se explicó en la sección 3.2.3.2, con la excepción de que en este caso los pesos iniciales de las capas convolucionales no son aleatorios, sino que corresponden a los pesos obtenidos al entrenar las redes DCGAN. Es decir que, antes de comenzar el entrenamiento, para el filtro de cada capa convolucional se copian los pesos del filtro de la capa análoga del discriminador DCGAN.

3.2.4.3. Resultados

Los mejores modelos se lograron con campo visual CE para 16:9 y ME para SQ, en ambos casos con 10 frames. De estos dos modelos, el que mejor *accuracy* obtuvo fue nuevamente el segundo, es decir, el que utiliza la relación de aspecto SQ.

Dicho modelo obtuvo los siguientes valores de *accuracy* para los conjuntos de datos de validación y prueba:

- **Validación:** $0,7782$
- **Prueba:** $0,7956$

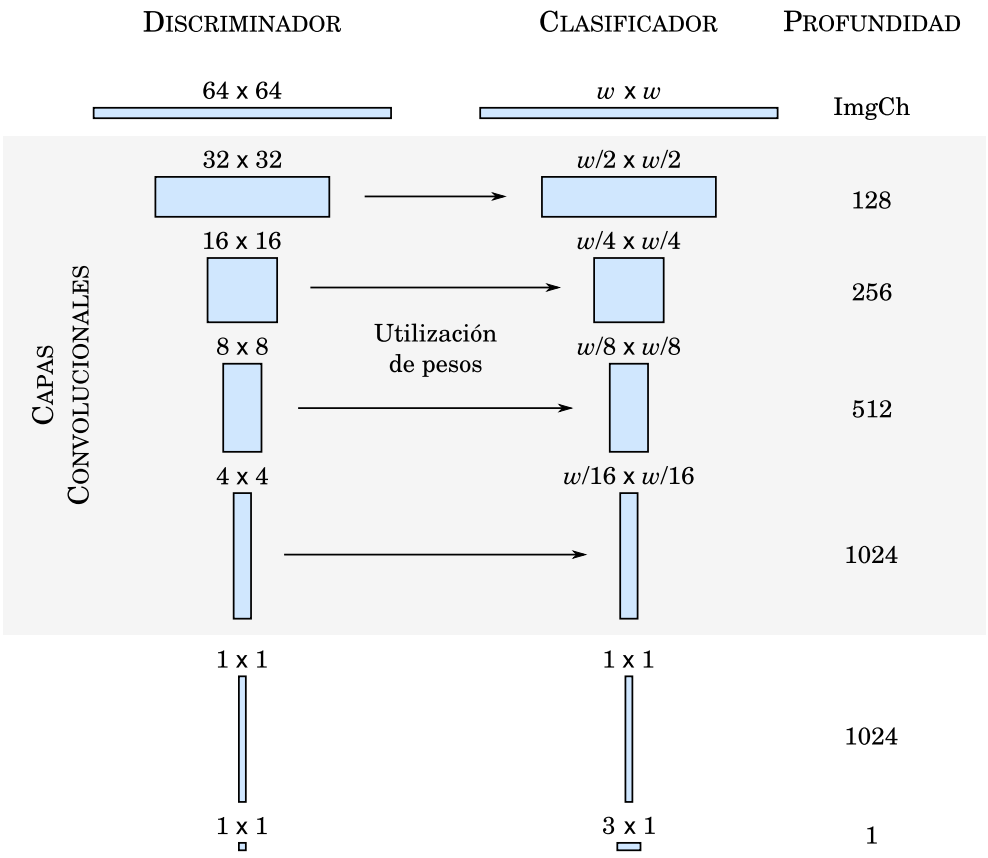


Figura 3.21: Comparación entre el discriminador de DCGAN y la red utilizada como clasificador. Comparten los filtros de las capas convolucionales.

Al igual que en la sección 3.2.3.3, se comparan los resultados obtenidos por los diferentes tamaños de campo visual, cantidades de frames y relaciones de aspecto. Para ello se mide la mejora promedio de la *accuracy* al emplear los diferentes valores:

Hiperparámetro	**Mejora *accuracy*** (%)
LE	$0,00 \pm 0,00$
ME	$0,87 \pm 0,45$
CE	$1,23 \pm 0,41$
1 frame	$0,00 \pm 0,00$
3 frames	$1,12 \pm 0,27$
5 frames	$1,36 \pm 0,43$
10 frames	$2,67 \pm 0,60$
16:9	$0,00 \pm 0,00$
SQ	$0,47 \pm 0,31$

Cuadro 3.14: Mejoras en la *accuracy* al modificar el tamaño de campo visual, la cantidad de frames o la relación de aspecto.

Finalmente, se comparan los resultados obtenidos por los modelos de las dos últimas secciones, es decir, las redes neuronales inicializadas de manera aleatoria por un lado, y las redes inicializadas con los pesos aprendidos por el discriminador de la sección 3.2.1.4 por otro. Esta comparación se realiza tanto con la relación de aspecto 16:9 como con la relación SQ, además de combinadas.

Rel. de aspecto	**Prom. mejora *accuracy*** (%)
16:9	$1,68 \pm 0,19$
SQ	$0,20 \pm 0,41$
Ambas	$0,94 \pm 0,27$

Cuadro 3.15: Promedios de las mejoras al utilizar los pesos aprendidos por la red GAN en lugar de inicializar la red aleatoriamente.

3.2.5. Generalización a videos completos

El origen de las ventanas con las que trabajamos en las secciones de aprendizaje supervisado es un conjunto de videos etiquetados. Al realizar las

preparaciones de los datasets para este trabajo (sección 3.1) no nos dedicamos a analizar el contenido de los mismos ni a realizar recortes sobre éstos para simplificarlos y hacer más fácil la tarea de clasificación.

Es por ello que estos videos, a pesar de estar correctamente etiquetados (una clase por video completo), no se limitan al momento exacto que define inequívocamente su clase. Por ejemplo, en un video etiquetado como *line* puede haber un tiempo previo de preparación de la jugada y, después de que ésta finaliza, puede haber parte de la jugada siguiente.

En esta sección intentaremos clasificar estos videos de forma completa. Es decir, en lugar de clasificar fragmentos de hasta 5 segundos como en las secciones anteriores, realizaremos predicciones tomando como muestra unitaria cada video e intentando determinar la clase a la que corresponde.

Entonces, el problema planteado consiste en utilizar los resultados de clasificación de fragmentos de videos que obtienen los modelos entrenados en las secciones anteriores a fin de obtener predicciones sobre videos completos. Utilizaremos el modelo que obtuvo menor error para el dataset de validación.

La forma más simple es contabilizar las predicciones individuales de todas las ventanas a lo largo del video, agrupadas por clase, y elegir la que obtuvo mayor cantidad de ventanas asociadas a la misma. Sin embargo, este método no contempla las posiciones de las ventanas, es decir que considera iguales a una ventana que se encuentra al principio del video, a una que se encuentra en el medio y a otra que se encuentra al final del mismo.

A continuación formalizaremos éste y otros métodos para transferir los resultados para ventanas a resultados para videos completos.

Sean DS el conjunto de todos los videos del dataset y V un video de este conjunto, compuesto por ventanas $\{v_0, v_1, ..., v_n\}$. Además, sea C el conjunto de clases del dataset, definido como

$$C = \{\textit{'line'}, \textit{'scrum'}, \textit{'juego'}\}.$$

Llamamos W al conjunto que contiene a todas las ventanas de todos los videos, es decir que

$$W = \bigcup_{V \in DS} V.$$

Ahora, sea h el modelo base utilizado, el cual asigna una clase $c \in C$ a cada ventana de cada video, por lo que es de la forma

$$h : W \longmapsto C.$$

Para simplificar las definiciones posteriores, y debido a que todos los métodos se basan en votaciones, definiremos una función genérica H que toma un video V, el modelo h predictor de ventanas, una función w de pesos, y retorna la clase predicha para V, utilizando a h como modelo base y los pesos de w.

La función $w(i, n)$ toma un índice $i \in \{0, 1, ..., n-1\}$ y la cantidad total de índices posibles n y retorna un peso o valor de ponderación para la i-ésima posición de un total de n posiciones temporales del video.

Antes de continuar, definimos dos funciones auxiliares:

$$Buscar(c, V, h) = \{\, i \mid h(V[i]) = c \,\},$$

donde $V[i]$ es la i-ésima ventana del video V. Esta función retorna las posiciones de las ventanas de V a las cuales el modelo h les asignó clase c.

$$Sumar(c, V, h, w) = \sum_{i \in Buscar(c,V,h)} w(i, n),$$

donde n es la cantidad de ventanas del video. Esta otra función retorna el puntaje total obtenido por la clase c para el video V, utilizando el modelo h y los valores de ponderación de w.

Finalmente, podemos definir H de la forma

$$H(V, h, w) = \arg\max_{c \in C} \; Sumar(c, V, h, w).$$

La diferencia entre los métodos probados consiste en utilizar diferentes funciones que asignan pesos a las ventanas de acuerdo a su posición, por lo que cada método quedará determinado por la elección de la función w.

3.2.5.1. Método con votación simple

En este caso, todas las ventanas son consideradas iguales, no hay ninguna que tenga más voto/peso que otra. Por ello la función w es constante, y se define

$$w(i, n) = 1.$$

3.2.5.2. Métodos con votación ponderada por ubicación temporal del fragmento

Como es posible que la jugada principal se desarrolle, por ejemplo, en el centro del video, al comienzo o al final, se prueban alternativas dando más peso a los votos provenientes de predicciones de ventanas de diferentes puntos del video.

- **Centro**

$$w_{centro}(i,n) = \begin{cases} i * 2/n, & \text{si } i \leq n/2 \\ -2i/n + 2, & \text{si } i > n/2 \end{cases}$$

- **Extremos**

$$w_{extremos}(i,n) = 1 - w_{centro}(i,n)$$

- **Comienzo**

$$w_{comienzo}(i,n) = -x/n + 1$$

- **Final**

$$w_{final}(i,n) = 1 - w_{comienzo}(i,n) = x/n$$

3.2.5.2.1. Ajuste

Los últimos cuatro métodos explicados asignan pesos que se encuentran en el intervalo $[0,1]$. Para atenuar el efecto de estos pesos (y aumentar ligeramente las alternativas abarcadas) se agregan los mismos métodos, pero añadiendo una constante $(0,5)$ a los pesos utilizados:

$$w'_{centro}(i,n) = w_{centro}(i,n) + 0,5$$

$$w'_{extremos}(i,n) = w_{extremos}(i,n) + 0,5$$

$$w'_{comienzo}(i,n) = w_{comienzo}(i,n) + 0,5$$

$$w'_{final}(i,n) = w_{final}(i,n) + 0,5$$

De esta manera, se reduce la diferencia (relativa) entre los pesos asignados a las ventanas con menor peso y las de mayor peso.

Para tener una mejor comprensión de los métodos definidos, en la Figura 3.22 graficamos las distintas variantes de la función w utilizadas.

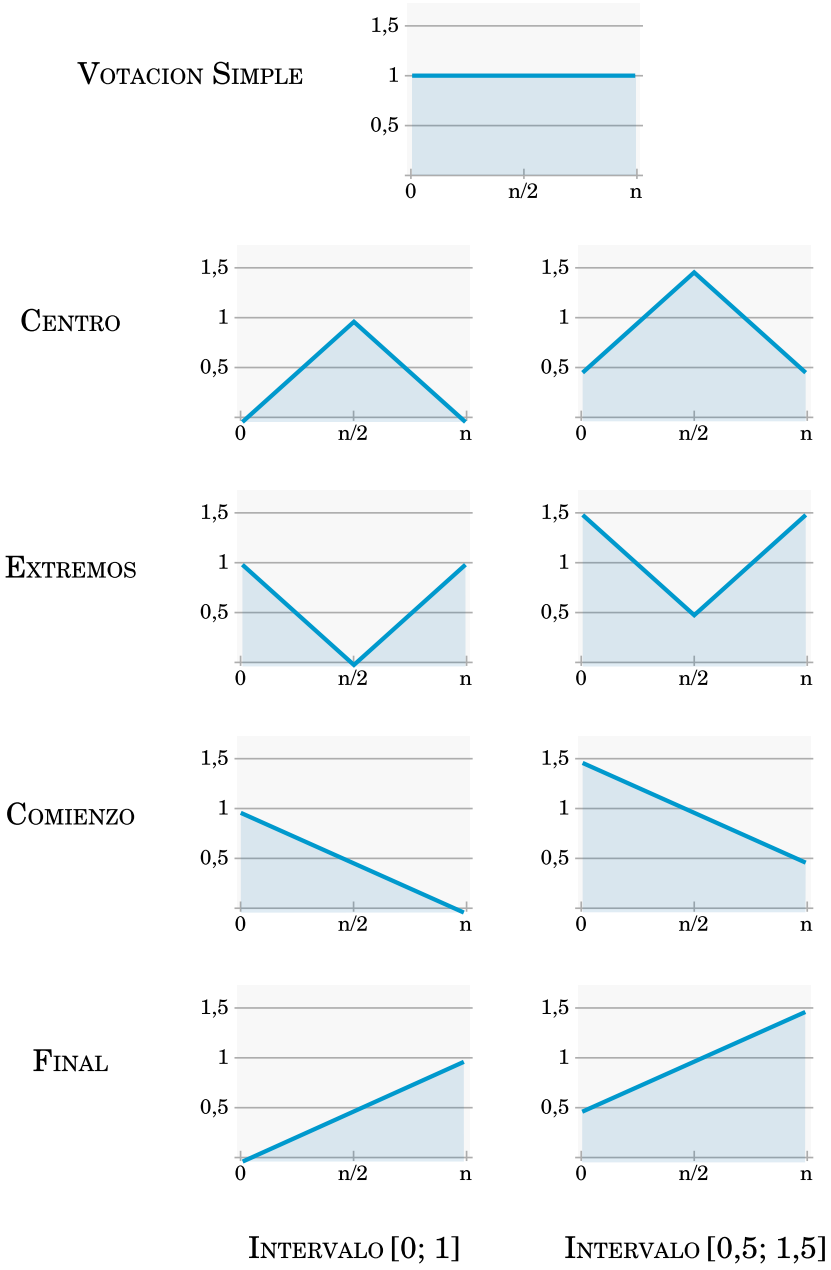


Figura 3.22: Gráficas de las funciones de pesos utilizadas en las votaciones.

3.2.5.3. Resultados

Las *accuracies* obtenidas por los métodos para el conjunto de datos de validación son las siguientes:

Método de votación		**Intervalo** $[0;\ 1]$	**Intervalo** $[0,5;\ 1,5]$
Simple		$0,6111$	—
Ponderada	Centro	$0,6111$	$0,6111$
	Extremos	$0,6111$	$0,6111$
	Comienzo	$\underline{0,6667}$	$\underline{0,6667}$
	Final	$0,6111$	$0,6111$

Cuadro 3.16: Resultados de *accuracy* por método.

El mejor puntaje lo obtuvieron los métodos que le dan mayor peso a las ventanas del comienzo del video ($w_{comienzo}$ y $w'_{comienzo}$). Dichos métodos de votación obtuvieron los siguientes resultados para los conjuntos de datos de validación y prueba (son iguales para ambos intervalos):

- **Validación:** $\dfrac{12}{18} \approx 0,6667$
- **Prueba:** $\dfrac{14}{18} \approx 0,7778$

Capítulo 4

Discusión y conclusiones

En este trabajo se experimentó con redes adversarias generativas para trabajar con videos. Se hizo mediante un abordaje naïve, al utilizar los canales de cada frame como si fueran todos canales de una misma imagen. En esta sección se encuentran las conclusiones finales a las que se arribó.

Como subproducto del proceso de entrenamiento de cada red GAN se obtuvieron un generador y un discriminador.

El generador mostró ser capaz de producir videos artificiales que, a pesar de ser distinguibles de los reales, en general contienen objetos similares a los presentes en las muestras reales. En algunos casos, éstos se desplazan de manera natural, con movimientos parecidos a los encontrados en las muestras originales (principalmente con campos visuales alejados y baja cantidad de frames). Trabajos anteriores [43] generaron secuencias de video en donde mayoritariamente los objetos cambian de forma, se desplazan y se fusionan de una manera irreal, pese a haber utilizado más de 12000 horas de videos preprocesados con estabilización de imagen y descarte de aquellos con características indeseables.

A partir del discriminador entrenado por la red se desarrolló una forma de generar vectores representación para secuencias de hasta 10 frames, los cuales fueron puestos a prueba al utilizarlos para realizar agrupamiento de muestras en base a ellos. En los resultados se encontró que los vectores estaban resumiendo información de alto nivel, como por ejemplo la presencia de árboles, carteles publicitarios, postes de rugby, cabeza y hombros de jugadores, situaciones en las que hay jugadores amontonados de forma similar a un scrum, a un line, etcétera. Previamente se hizo agrupamiento utilizando sólo histogramas sobre los componentes de PCA, aplicado directamente sobre los

píxeles de las imágenes, en donde se vio que las muestras se agrupan utilizando información de bajo nivel, como por ejemplo la luminosidad global, luminosidad por zonas, campo de juego (superficie verde), cielo o vestimentas celestes, imágenes globalmente azuladas o rojizas, entre otros.

Los mismos vectores se utilizaron luego para clasificar jugadas deportivas, recortando parches de diferentes tamaños y duraciones de los videos, generando sus respectivos vectores representación y entrenando SVMs con diferentes configuraciones. Los mejores resultados se lograron empleando ventanas de tamaño de campo visual LE (lejos) y 10 frames de duración.

El mismo problema de clasificación se abordó utilizando redes convolucionales profundas, las cuales tomaron como entrada frames completos en lugar de recortes, pero mantuvieron las mismas duraciones que el enfoque con SVMs. Los mejores resultados se obtuvieron nuevamente utilizando ventanas con 10 frames de duración, pero el mejor campo visual esta vez resultó ser CE (cerca). Además, debieron considerarse dos formas distintas de adaptar las muestras no cuadradas para utilizarlas en redes con entradas cuadradas, resultando vencedora la que consiste en estirar la imagen original en una de sus dimensiones para ocupar toda la superficie de entrada (perdiendo la relación de aspecto original). El enfoque con redes obtuvo un error de validación menor que aquel con SVMs.

Posteriormente, al uso de redes convolucionales profundas se le adicionó la información aprendida por las GANs. Esto se hizo trasladando los pesos aprendidos por el clasificador durante su entrenamiento para utilizarlos como pesos iniciales de estas redes, buscando mejorar los resultados para el mismo problema de clasificación. Las ventanas con 10 frames fueron otra vez las que obtuvieron mejores resultados, en esta oportunidad con tamaño de campo visual ME (medio). Esta modificación logró disminuir el error de validación, comparado con el obtenido previamente por las redes inicializadas con pesos aleatorios.

Nuevamente, los mejores resultados se presentaron con el método que estira las imágenes de entrada, aunque en esta ocasión la mejora promedio fue más baja que en el caso anterior. Esta diferencia puede deberse a que las redes GAN fueron entrenadas con ventanas de imágenes que mantuvieron su relación de aspecto original.

A lo largo de todo el trabajo se utilizaron diferentes cantidades de frames y tamaños de campos visuales, con la idea de encontrar las variantes que mejor funcionan dentro de las limitaciones de los modelos utilizados. Estas son las conclusiones que podemos sacar sobre los valores utilizados para dichos

hiperparámetros.

Tamaño de campo visual Al utilizar SVMs, el error desciende a medida que ampliamos el campo visual (alejamos la cámara), esto puede deberse a que estamos analizando una superficie mayor de la imagen (recordemos que se utiliza una grilla con nueve recortes, los cuales para campos visuales cercanos no se superponen). Por el contrario, al utilizar redes el error decrece al reducir el campo visual (acercar la cámara). Esto es razonable considerando que la red analiza la totalidad de la imagen, por lo que al reducir el campo visual estamos ampliando los detalles que puede percibir y codificar en capas intermedias sin perder información, como ocurría con las grillas utilizadas para generar las muestras utilizadas en las SVMs.

Cantidad de frames por muestra En todos los experimentos se mantuvo la tendencia de que al aumentar la cantidad de frames disminuye el error, siendo 10 frames el valor que obtuvo mejores resultados para todos los algoritmos utilizados en este trabajo.

Es importante destacar que durante el entrenamiento de las redes GAN, en repetidas ocasiones se produjo una desestabilización de los pesos, de la cual el algoritmo de entrenamiento no siempre pudo recuperarse completamente. En estos casos se observó que las imágenes de control generadas durante las iteraciones con problemas poseían características indeseables que las diferenciaban claramente de las muestras reales, como ser bajo contraste (imágenes que tienden considerablemente al gris) e imágenes casi iguales para diferentes vectores de ruido aplicados en la entrada del generador. Estas alteraciones aparecieron con mayor frecuencia cuando las muestras tenían mayor cantidad de frames o un campo visual más alejado (más objetos y más pequeños). Los efectos negativos de esto pudieron evitarse en cierta medida al detener el entrenamiento y utilizar los parámetros guardados en un punto anterior a la aparición del problema.

Finalmente, se analizaron algunos métodos para extrapolar los resultados de clasificación de secuencias cortas hacia videos completos. Por un lado se utilizó un método que asigna el mismo peso a cada voto y por otro, uno que asigna distintos pesos a los mismos, en función de la posición que ocupa la secuencia de frames dentro del video. Pese a que uno de los métodos (votación ponderada, con mayor peso al comienzo del video) obtuvo resultados levemente superiores al resto, en general para todos los métodos se dieron

resultados casi iguales. Esto se puede deber a que un sistema de votación es demasiado simple y hace algunas suposiciones implícitas, como que todas las clases "son lo mismo". Por ejemplo, asume que tiene el mismo peso o importancia que un fragmento del video se clasifique como '*scrum*' o que se clasifique como '*juego*', cuando la lógica nos indica que si en unos pocos frames se reconoce un scrum (por su formación característica) es casi seguro que la jugada completa se trata de eso (aunque en el resto de la misma no se detecte tal formación), mientras que si en ese fragmento no se detecta un scrum ni un line (es decir, se clasifica como '*juego*') no se puede tener mucha certeza de que esa sea la clase de la jugada, ya que encontrar características de dicha clase no es algo tan contundente.

Capítulo 5

Trabajos futuros

A continuación se plantean trabajos futuros, algunos corresponden a posibles ampliaciones de este trabajo y el resto a líneas de investigación alternativas:

1. Visualizar y analizar los filtros de la primera capa convolucional del discriminador.
2. Analizar el efecto producido en los videos sintetizados por el generador al variar componentes del vector de ruido utilizado en su entrada.
3. Mejorar las predicciones de videos completos, utilizando algún algoritmo de clasificación que se entrene utilizando los datos.
4. Extender el trabajo a otros deportes o con nuevas acciones de rugby.
5. Utilizar Wasserstein GAN (WGAN), una alternativa a las GANs tradicionales que mejora la estabilidad de aprendizaje [44, 45].
6. Emplear GANs en las cuales el generador y el discriminador realizan convoluciones 3D.

Bibliografía

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, ..., and Y. Bengio. Generative adversarial nets. *Advances in neural information processing systems*, pages 2672–2680, 2014.

[2] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

[3] P. Isola, J. Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. *arXiv preprint*, 2017.

[4] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, and W. ... Shi. Photo-realistic single image super-resolution using a generative adversarial network. *arXiv preprint*, 2016.

[5] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee. Generative adversarial text to image synthesis. *arXiv preprint arXiv:1605.05396*, 2016.

[6] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2536–2544, 2016.

[7] J. Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. *arXiv preprint arXiv:1703.10593*, 2017.

[8] S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach.* Prentice Hall, 2010.

[9] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2016.

[10] E. Alpaydin. *Introduction to Machine Learning*. MIT Press, 2014.

[11] Google LLC. Glosario sobre aprendizaje automático, 2018. Obtenido de `https://developers.google.com/machine-learning/crash-course/glossar`.

[12] V. Vapnik. *Statistical Learning Theory*. Wiley Interscience, 1998.

[13] N. Cristianini and J. Shawe-Taylor. *An introduction to Support Vector Machines*. Cambridge University Press, 2000.

[14] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. Obtenido de `https://www.deeplearningbook.org/`.

[15] L. Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010.

[16] Google LLC. Reducción de la pérdida: Descenso de gradiente estocástico, 2018. Obtenido de `https://developers.google.com/machine-learning/crash-course/reducing-loss/stochastic-gradient-descent`.

[17] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

[18] S. Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

[19] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[20] G. E. Hinton, S. Osindero, and Y. Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.

[21] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.

[22] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639):115, 2017.

[23] Y. Yuan, M. Chao, and Y. Lo. Automatic skin lesion segmentation using deep fully convolutional networks with jaccard distance. *IEEE Trans. Med. Imaging*, 36(9):1876–1886, 2017.

[24] M. Havaei, A. Davy, D. Warde-Farley, A. Biard, A. Courville, Y. Bengio, C. Pal, P. Jodoin, and H. Larochelle. Brain tumor segmentation with deep neural networks. *Medical image analysis*, 35:18–31, 2017.

[25] J. Li, X. Mei, D. Prokhorov, and D. Tao. Deep neural network for structural prediction and lane detection in traffic scene. *IEEE transactions on neural networks and learning systems*, 28(3):690–703, 2017.

[26] X. Du, M. El-Khamy, J. Lee, and L. Davis. Fused dnn: A deep neural network fusion approach to fast and robust pedestrian detection. In *Applications of Computer Vision (WACV), 2017 IEEE Winter Conference on*, pages 953–961. IEEE, 2017.

[27] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[28] Q. Yu, Y. Yang, F. Liu, Y. Song, T. Xiang, and T. M. Hospedales. Sketch-a-net: A deep neural network that beats humans. *International journal of computer vision*, 122(3):411–425, 2017.

[29] A. Karpathy and L. Fei-Fei. Deep visual-semantic alignments for generating image descriptions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3128–3137, 2015.

[30] S. Iizuka, E. Simo-Serra, and H. Ishikawa. Let there be color!: joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification. *ACM Transactions on Graphics (TOG)*, 35(4):110, 2016.

[31] S. Suwajanakorn, S. M. Seitz, and I. Kemelmacher-Shlizerman. Synthesizing obama: learning lip sync from audio. *ACM Transactions on Graphics (TOG)*, 36(4):95, 2017.

[32] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation, 1(4)*, pages 541–551, 1989.

[33] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[34] A. Karpathy. Convolutional neural networks (cnns / convnets). *Notes from the Stanford CS class CS231n*, 2016. Obtenido de `https://cs231n.github.io/convolutional-networks/`.

[35] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

[36] J. Ngiam, Z. Chen, D. Chia, P. W. Koh, Q. V. Le, and A. Y. Ng. Tiled convolutional neural networks. *Advances in neural information processing systems*, pages 1279–1287, 2010.

[37] R. Girshick. Object detection, deep learning, and r-cnns [diapositivas], 2014. Obtenido de `https://homes.cs.washington.edu/~shapiro/EE596/notes/DeepLearning.pdf`.

[38] V. Dumoulin and F. Visin. A guide to convolution arithmetic for deep learning. *arXiv preprint arXiv:1603.07285*, 2016.

[39] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.

[40] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. Improved techniques for training gans. *Advances in Neural Information Processing Systems*, pages 2234–2242, 2016.

[41] S. Thalles. A short introduction to generative adversarial networks, 2017. Obtenido de `https://sthalles.github.io/intro-to-gans/`.

[42] World Rugby. *Leyes del Juego de Rugby*. Obtenido de `https://laws.worldrugby.org/`.

[43] C. Vondrick, H. Pirsiavash, and A. Torralba. Generating videos with scene dynamics. In *Advances In Neural Information Processing Systems*, pages 613–621, 2016.

[44] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein gan. *arXiv preprint arXiv:1701.07875*, 2017.

[45] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, pages 5767–5777, 2017.

Apéndice A

Generación de imágenes artificiales

A.1. Calidad de las imágenes

Se muestra sólo el primer frame, ya que el objetivo de este conjunto de imágenes es apreciar la calidad de las mismas, sin importar la generación de movimiento.



Figura A.1: Muestra de datos reales; CE; 1 frame.



Figura A.2: Muestra de datos artificiales; CE; 1, 3, 5 y 10 frames.



Figura A.3: Muestra de datos reales; ME; 1 frame.



Figura A.4: Muestra de datos artificiales; ME; 1, 3, 5 y 10 frames.



Figura A.5: Muestra de datos reales; LE; 1 frame.



Figura A.6: Muestra de datos artificiales; LE; 1, 3, 5 y 10 frames.

A.2. Movimiento de las imágenes

El eje vertical de las imágenes presentes en esta sección indica el tiempo, es decir, el número de frame. El eje horizontal, por otro lado, indica la muestra generada.

Además de las imágenes mostradas aquí, se pueden ver videos generados durante los experimentos de este trabajo en `https://dcc.fceia.unr.edu.ar/~lchehem/`.



Figura A.7: Muestra de datos artificiales; CE; 3, 5 y 10 frames.



Figura A.8: Muestra de datos artificiales; ME; 3, 5 y 10 frames.



Figura A.9: Muestra de datos artificiales; LE; 3, 5 y 10 frames.

A.3. Entrenamiento con pérdida de contraste

En las siguientes figuras, en la última columna se encuentra el color promedio de las imágenes de esa iteración del entrenamiento, junto con un indicador de su saturación promedio.



Figura A.10: Entrenamiento con imágenes que pierden contraste, 1ra parte.



Figura A.11: Entrenamiento con imágenes que pierden contraste, 2da parte.

Apéndice B

Agrupamiento de imágenes utilizando vectores representación

Las descripciones de los clusters corresponden a la mayoría de las imágenes que lo componen, pero todos los clusters tienen algunas imágenes que no encajan en su descripción. Además, algunas descripciones pueden ser subjetivas.



Figura B.1: De izquierda a derecha, imágenes tomadas al azar de cada uno de los diez bins del histograma generado con el componente 0 de PCA aplicado a los vectores producidos con campo visual CE y muestras de 1 frame.



Figura B.2: Imágenes de un cluster formado utilizando el componente 3 de PCA, con campo visual CE y muestras con 1 frame. Las imágenes contienen árboles en su parte inferior y cielo en su parte superior.

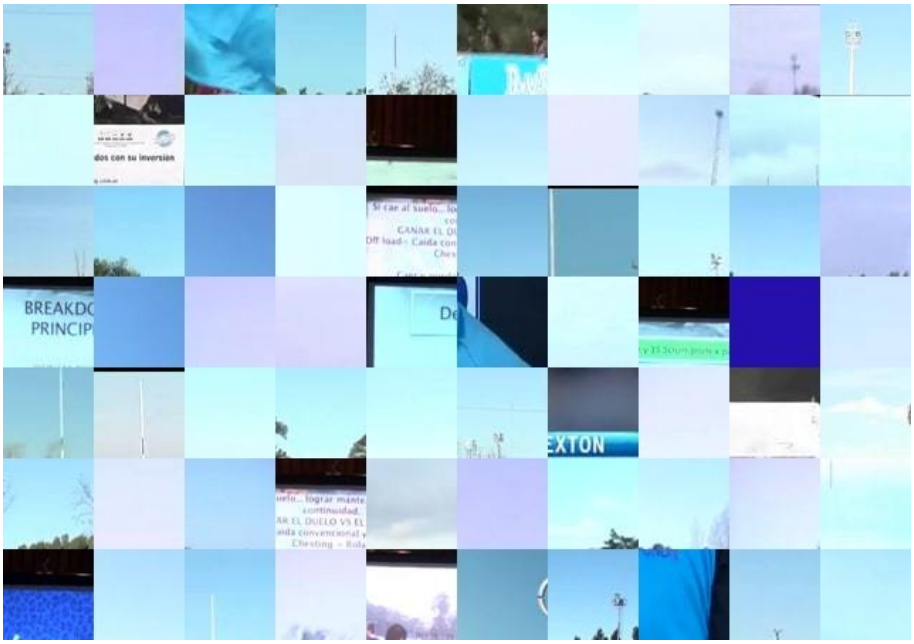


Figura B.3: Imágenes de un cluster formado utilizando el componente 4 de PCA, con campo visual CE y muestras con 1 frame. Contiene imágenes celestes o azules, en su mayoría homogéneas, correspondientes al cielo.



Figura B.4: Imágenes de un cluster formado utilizando el componente 5 de PCA, con campo visual CE y muestras con 1 frame. Compuesto por imágenes cuyo color predominante es el azul. No son imágenes homogéneas. Una parte importante de ellas presenta postes o caños.



Figura B.5: Imágenes de un cluster formado utilizando el componente 7 de PCA, con campo visual CE y muestras con 1 frame. Presenta imágenes con bordes horizontales, que cubren casi todo el ancho de las mismas.



Figura B.6: Imágenes de un cluster formado utilizando el componente 6 de PCA, con campo visual CE y muestras con 1 frame. Compuesto por imágenes con carteles publicitarios con colores saturados, mayormente azul y rojo.



Figura B.7: Imágenes de un cluster formado utilizando el componente 8 de PCA, con campo visual CE y muestras con 1 frame. Casi todas las imágenes contienen postes de rugby.



Figura B.8: Imágenes de un cluster formado utilizando el componente 9 de PCA, con campo visual CE y muestras con 1 frame. Contiene muchas imágenes en cuya sección izquierda (y mayormente inferior) predomina el color blanco o similar.



Figura B.9: Imágenes de un cluster formado utilizando el componente 5 de PCA, con campo visual CE y muestras con 3 frames. Captura imágenes cuyo centro es blanco, principalmente son carteles de publicidad.



Figura B.10: Imágenes de un cluster formado utilizando el componente 8 de PCA, con campo visual CE y muestras con 3 frames. Casi todas contienen la copa de un árbol, ocupando la parte derecha de la imagen, mostrando un poco de cielo en su sección izquierda.



Figura B.11: Imágenes de un cluster formado utilizando el componente 3 de PCA, con campo visual CE y muestras con 3 frames. Imágenes complejas, con muchos objetos u objetos con gran cantidad de detalles.



Figura B.12: Imágenes de un cluster formado utilizando el componente 7 de PCA, con campo visual CE y muestras con 3 frames. Contienen bordes horizontales, principalmente sombras en el campo de juego, tribunas vacías y carteles publicitarios.



Figura B.13: Imágenes de un cluster formado utilizando el componente 4 de PCA, con campo visual ME y muestras con 1 frame. Casi todas las imágenes presentan como colores predominantes el amarillo y el negro.



Figura B.14: Imágenes de un cluster formado utilizando el componente 6 de PCA, con campo visual ME y muestras con 1 frame. La mayoría contiene carteles de color blanco o similar, en su parte superior.



Figura B.15: Imágenes de un cluster formado utilizando el componente 1 de PCA, con campo visual LE y muestras con 1 frame. Este cluster captura imágenes que, en su mayoría, contienen cabeza y pecho de un jugador, generalmente vistiendo camisetas de colores saturados.



Figura B.16: Imágenes de un cluster formado utilizando el componente 3 de PCA, con campo visual LE y muestras con 1 frame. Contienen grupos de jugadores que podrían estar participando de un scrum.



Figura B.17: Imágenes de un cluster formado utilizando el componente 5 de PCA, con campo visual LE y muestras con 1 frame. Presentan en su mayoría cabeza y hombros de un jugador, ya sea de frente, de costado o de espaldas a la cámara.



Figura B.18: Imágenes de un cluster formado utilizando el componente 8 de PCA, con campo visual LE y muestras con 1 frame. Compuestas por grupos de jugadores que podrían estar participando de un scrum o line. Ubicados mayormente en la franja horizontal central levemente inferior.



Figura B.19: Imágenes de un cluster formado utilizando el componente 7 de PCA, con campo visual LE y muestras con 10 frames. Contienen jugadores que podrían estar participando de un scrum o line.



Figura B.20: Imágenes de un cluster formado utilizando el componente 8 de PCA, con campo visual LE y muestras con 10 frames. Compuestas en su mayoría por jugadores con muy poca separación entre ellos, ubicados ligeramente en la sección central izquierda de la imagen.